Mining Software Repositories

Chapter 2: Collection of raw data

Prof. Dr. Steffen Herbold

Contents

- Overview of data sources
  - Source code and version control
  - Issue tracker
  - Code review tools
  - Continuous integration systems
  - Artifact repositories
  - Forges
  - Communication tools
  - Wikis
  - Q&A and developer websites
- Tools for data collection
  - Examples for existing tools
  - Building your own tools
- Reusing data

Overview of data sources

Infeasible to list all data sources

- Every tool that any developer uses that records data is a potential MSR data source

- Which data sources are relevant should be problem-driven:
  - First determine what you want to know or achieve, then determine what you need to achieve this
  - Not: this data is conveniently available for me, what can I do with this
  - In reality often a combination of both

- Nevertheless, some data sources are very often very useful
  - E.g., relevant for many use cases or research questions

- Raw data means unprocessed data
  - Linking between data sources, heuristic enrichment of data, or human annotation are covered in later chapters

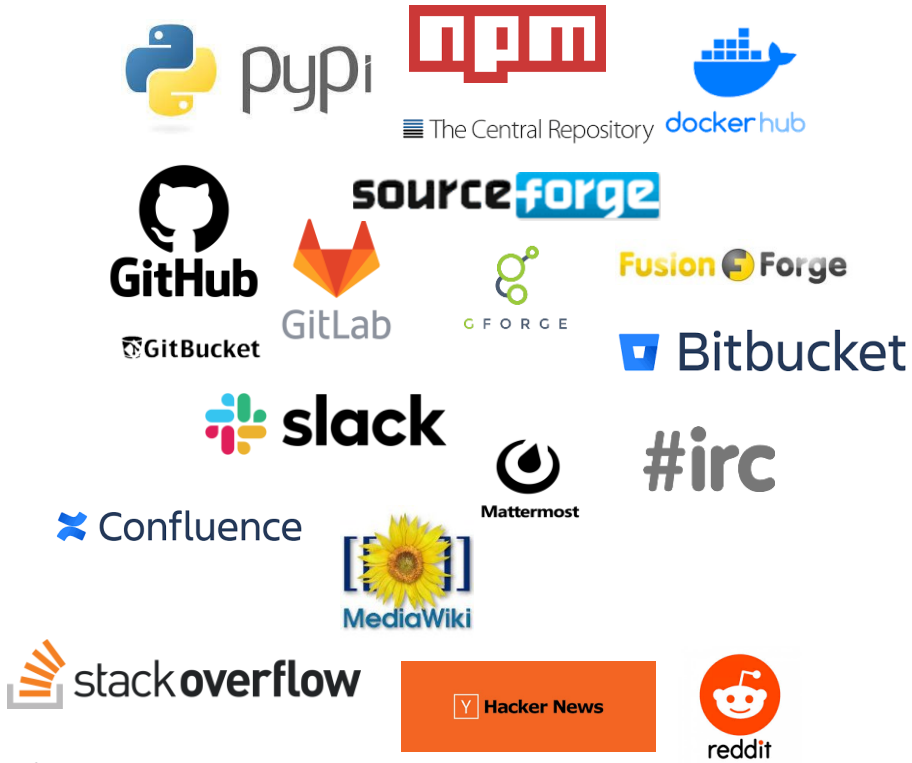
A large and incomplete list of data sources grouped by type of data (1/2)

- Source code
  - Single files, classes, whole code base
  - Code with application logic (e.g., Java, Python)
  - Code for building software (e.g., Make, gradle)
  - Code for deployments (e.g., Dockerfiles)
  - Code for configuring build systems (e.g., GitHub action files)
- Version control systems
  - Git, SVN, CVS, Mercurial, BitKeeper, Bazaar, Visual SourceSafe, RCS
- Issue tracking systems
  - Bugzilla, Jira, Launchpad, Redmine, Trac, GitHub Issues, GitLab Issues, …
- Code review tools
  - Gerrit, Crucible, Phabricators, GitHub within pull requests, GitLab within merge requests, …
- Continuous integration systems
  - Travis CI, Circle CI, Jenkins, GitHub actions, GitLab runners, …



A large and incomplete list of data sources grouped by type of data (2/2)

- Artifact repositories
  - Pypi, npm, Maven central, GitHub packages, Dockerhub, …
- Forges
  - GitHub, GitLab, SourceForge, GForge, FusionForge, BitBucket, GitBucket, …
- Communication tools
  - Mailing lists, IRC, Slack, Mattermost, GitHub discussions, Google Groups, …
- Wikis
  - GitHub wikis, Confluence, MediaWiki, …
- Q&A sites and developer forums
  - Stack Exchange (esp. Stack Overflow), HackerNews, some sub-reddits
- … and MORE!
  - Requirements engineering tools, test management tools, project management tools, …
- We will now show for each type what data can be collected
  - But not show this for all systems we mention as example!



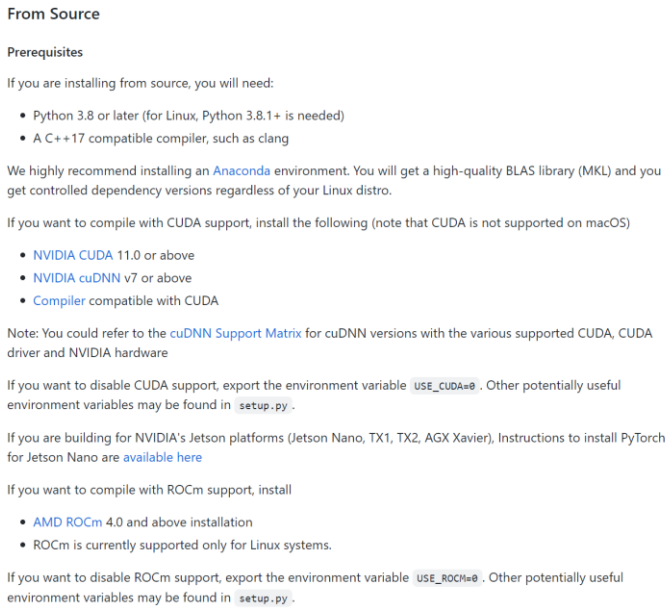
Source code and version control

Mining source code

- Source code
  - Every source code file is an artifact that can be analyzed
    - Size, complexity, readability, data flow, external dependencies
  - Every project is an artifact that can be analyzed
    - Similar to single file, but aggregated over whole project
    - Aspects on project level, like use of dependencies
- Source code is not only the programming, but also build configurations etc.
  - Build files contain information dependencies
  - Continuous integration files contain information regarding publishing and deployment

Working with source code is hard!

- Example: building the latest release of PyTorch (state March 2023)



From Source

Prerequisites

If you are installing from source, you will need:

- Python 3.8 or later (for Linux, Python 3.8.1+ is needed)
- A C++17 compatible compiler, such as clang

We highly recommend installing an Anaconda environment. You will get a high-quality BLAS library (MKL) and you get controlled dependency versions regardless of your Linux distro.

If you want to compile with CUDA support, install the following (note that CUDA is not supported on macOS)

- NVIDIA CUDA 11.0 or above
- NVIDIA cuDNN v7 or above
- Compiler compatible with CUDA

Note: You could refer to the cuDNN Support Matrix for cuDNN versions with the various supported CUDA, CUDA driver and NVIDIA hardware

If you want to disable CUDA support, export the environment variable `USE_CUDA=0`. Other potentially useful environment variables may be found in `setup.py`.

If you are building for NVIDIA's Jetson platforms (Jetson Nano, TX1, TX2, AGX Xavier), Instructions to install PyTorch for Jetson Nano are available here

If you want to compile with ROCm support, install

- AMD ROCm 4.0 and above installation
- ROCm is currently supported only for Linux systems.

If you want to disable ROCm support, export the environment variable `USE_ROCM=0`. Other potentially useful environment variables may be found in `setup.py`.



Install Dependencies

Common

```
conda install cmake ninja
# Run this command from the PyTorch directory after cloning the source code using the "Get the PyTorch Source
pip install -r requirements.txt
```

On Linux

```
conda install mkl mkl-include
# CUDA only: Add LAPACK support for the GPU if needed
conda install -c pytorch magma-cuda110  # or the magma-cuda* that matches your CUDA version from https://anac
```

On MacOS

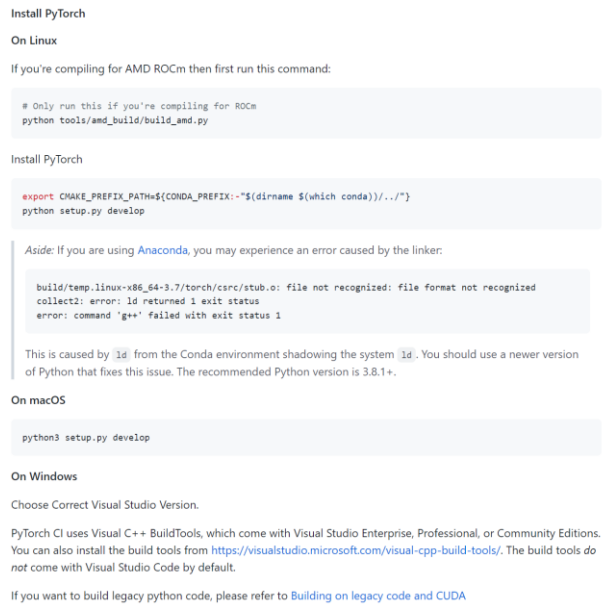
```
# Add this package on intel x86 processor machines only
conda install mkl mkl-include
# Add these packages if torch.distributed is needed
conda install pkg-config libuv
```

On Windows

```
conda install mkl mkl-include
# Add these packages if torch.distributed is needed.
# Distributed package support on Windows is a prototype feature and is subject to changes.
conda install -c conda-forge libuv=1.39
```

Get the PyTorch Source

```
git clone --recursive https://github.com/pytorch/pytorch
cd pytorch
# If you are updating an existing checkout
git submodule sync
git submodule update --init --recursive
```



Install PyTorch

On Linux

If you're compiling for AMD ROCm then first run this command:

```
# Only run this if you're compiling for ROCm
python tools/amd_build/build_amd.py
```

Install PyTorch

```
export CMAKE_PREFIX_PATH=${CONDA_PREFIX:-"$(dirname $(which conda))/../"}
python setup.py develop
```

> *Aside:* If you are using Anaconda, you may experience an error caused by the linker:
>
> ```
> build/temp.linux-x86_64-3.7/torch/csrc/stub.o: file not recognized: file format not recognized
> collect2: error: ld returned 1 exit status
> error: command 'g++' failed with exit status 1
> ```
>
> This is caused by `ld` from the Conda environment shadowing the system `ld`. You should use a newer version of Python that fixes this issue. The recommended Python version is 3.8.1+.

On macOS

```
python3 setup.py develop
```

On Windows

Choose Correct Visual Studio Version.

PyTorch CI uses Visual C++ BuildTools, which come with Visual Studio Enterprise, Professional, or Community Editions. You can also install the build tools from https://visualstudio.microsoft.com/visual-cpp-build-tools/. The build tools *do not* come with Visual Studio Code by default.

If you want to build legacy python code, please refer to Building on legacy code and CUDA



CPU-only builds

In this mode PyTorch computations will run on your CPU, not your GPU

```
conda activate
python setup.py develop
```

Note on OpenMP: The desired OpenMP implementation is Intel OpenMP (iomp). In order to link against iomp, you'll need to manually download the library and set up the building environment by tweaking `CMAKE_INCLUDE_PATH` and `LIB`. The instruction here is an example for setting up both MKL and Intel OpenMP. Without these configurations for CMake, Microsoft Visual C OpenMP runtime (vcomp) will be used.

CUDA based build

In this mode PyTorch computations will leverage your GPU via CUDA for faster number crunching

NVTX is needed to build Pytorch with CUDA. NVTX is a part of CUDA distributive, where it is called "Nsight Compute". To install it onto an already installed CUDA run CUDA installation once again and check the corresponding checkbox. Make sure that CUDA with Nsight Compute is installed after Visual Studio.

Currently, VS 2017 / 2019, and Ninja are supported as the generator of CMake. If `ninja.exe` is detected in `PATH`, then Ninja will be used as the default generator, otherwise, it will use VS 2017 / 2019.
If Ninja is selected as the generator, the latest MSVC will get selected as the underlying toolchain.

Additional libraries such as Magma, oneDNN, a.k.a MKLDNN or DNNL, and Sccache are often needed. Please refer to the installation-helper to install them.

You can refer to the build_pytorch.bat script for some other environment variables configurations

```
cmd

:: Set the environment variables after you have downloaded and unzipped the mkl package,
:: else CMake would throw an error as `Could NOT find OpenMP`.
set CMAKE_INCLUDE_PATH={Your directory}\mkl\include
set LIB={Your directory}\mkl\lib;%LIB%

:: Read the content in the previous section carefully before you proceed.
:: [Optional] If you want to override the underlying toolset used by Ninja and Visual Studio with CUDA, pleas
:: "Visual Studio 2019 Developer Command Prompt" will be run automatically.
:: Make sure you have CMake >= 3.12 before you do this when you use the Visual Studio generator.
set CMAKE_GENERATOR_TOOLSET_VERSION=14.27
set DISTUTILS_USE_SDK=1
for /f "usebackq tokens=*" %i in (`"%ProgramFiles(x86)%\Microsoft Visual Studio\Installer\vswhere.exe" -versi

:: [Optional] If you want to override the CUDA host compiler
set CUDAHOSTCXX=C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Tools\MSVC\14.27.29110\bin\H

python setup.py develop
```

and we have not done an analysis yet …

https://github.com/pytorch/pytorch#from-source

Static source code analysis

- Analysis of the code without executing the code
- Often does not require building the code
  - But this depends! Compilers are also doing static analysis!
- Can work directly on the text
  - E.g., lines of code, identification of patterns using regular expressions
- Can require the creation of an Abstract Syntax Tree (AST)
  - Requires parsing the code
  - Can often be done independently for each file
    - Greatly reduces the complexity, but depends! Not possible, e.g., for C++ code.
- Can require the creation of a control flow or data flow graph
  - Goes beyond AST parsing
  - Allows deeper analysis of static patterns, e.g., for finding anomalies like dead code or unitialized variables

Approaches for source code analysis (cnt.)

- Dynamic analysis
  - Requires building and executing the source code
  - Allows observing software behavior directly
    - E.g., through software tests
  - Enables otherwise infeasible analysis
    - E.g., energy consumption
  - Building software retrospectively is very difficult!
    - E.g., research found that on average only 38% of Maven snapshots from the Apache Foundation can still be build without additional effort

→ Because dynamic analysis is so difficult, most MSR studies rely on static analysis

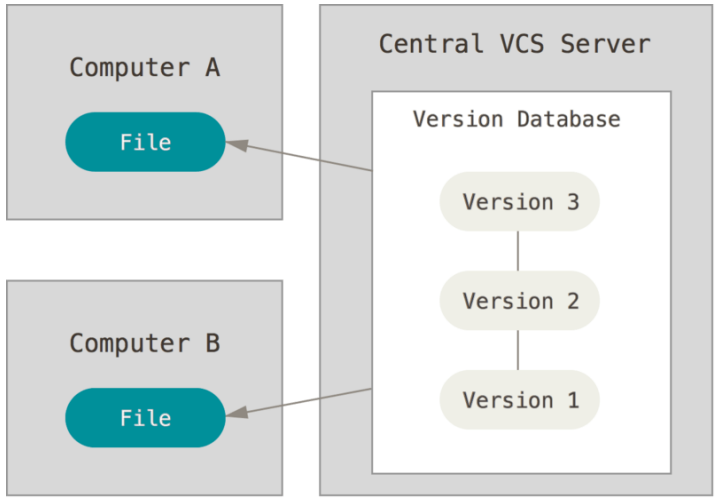
Version Control Systems (VCS)

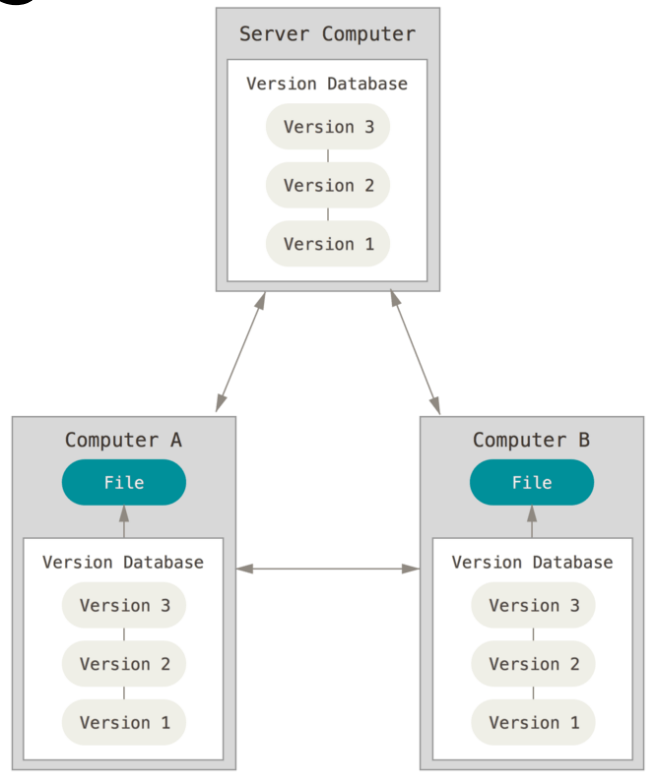
- Versioning of software vital for software development
  - Allows tracking and potential reversal of changes
  - Allows parallel development and merging of changes

- VCSs enable this
- Git is the currently most popular, but others still in use!

- Git is only the one tool in a long history of VCSs. Some notable examples are:
  - Concurrent Version System (CVS) – released 1990
    - Allowed tracking of changes to files, but only one file could be changed at a time
  - Subversion (SVN) – released 2000
    - Similar to CVS, but e.g., allowed changing many files at once
  - Mercurial – released 2005
    - Distributed version control system, similar to Git
- We focus on Git in this lecture – but most aspects translate to other systems

Central vs decentralized VCS



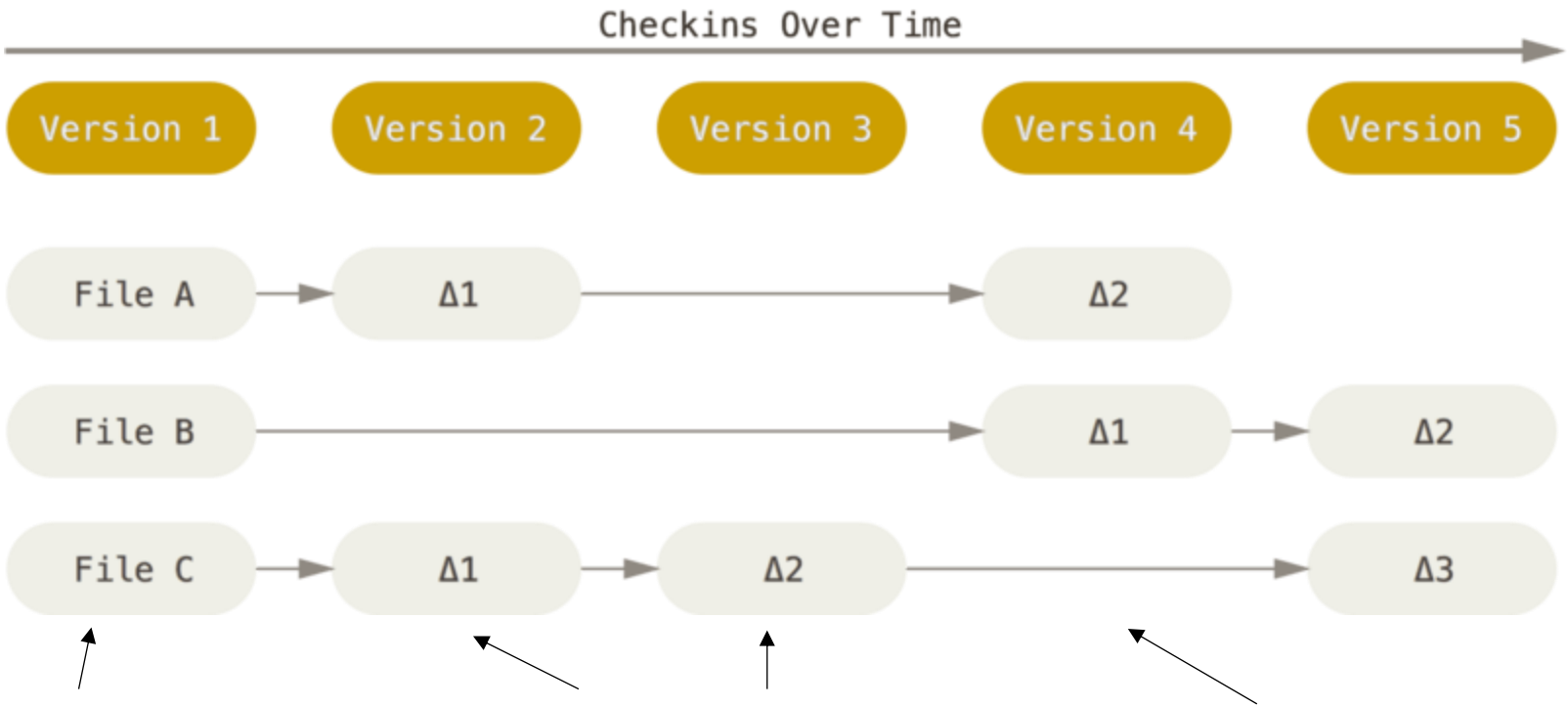


Centralized version control: a server contains the version data base and clients only keep local copies of the current state

Decentralized version control: clients and servers all keep full copies of the version control database and need to agree on which is the current state

https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Sequences of deltas / diffs

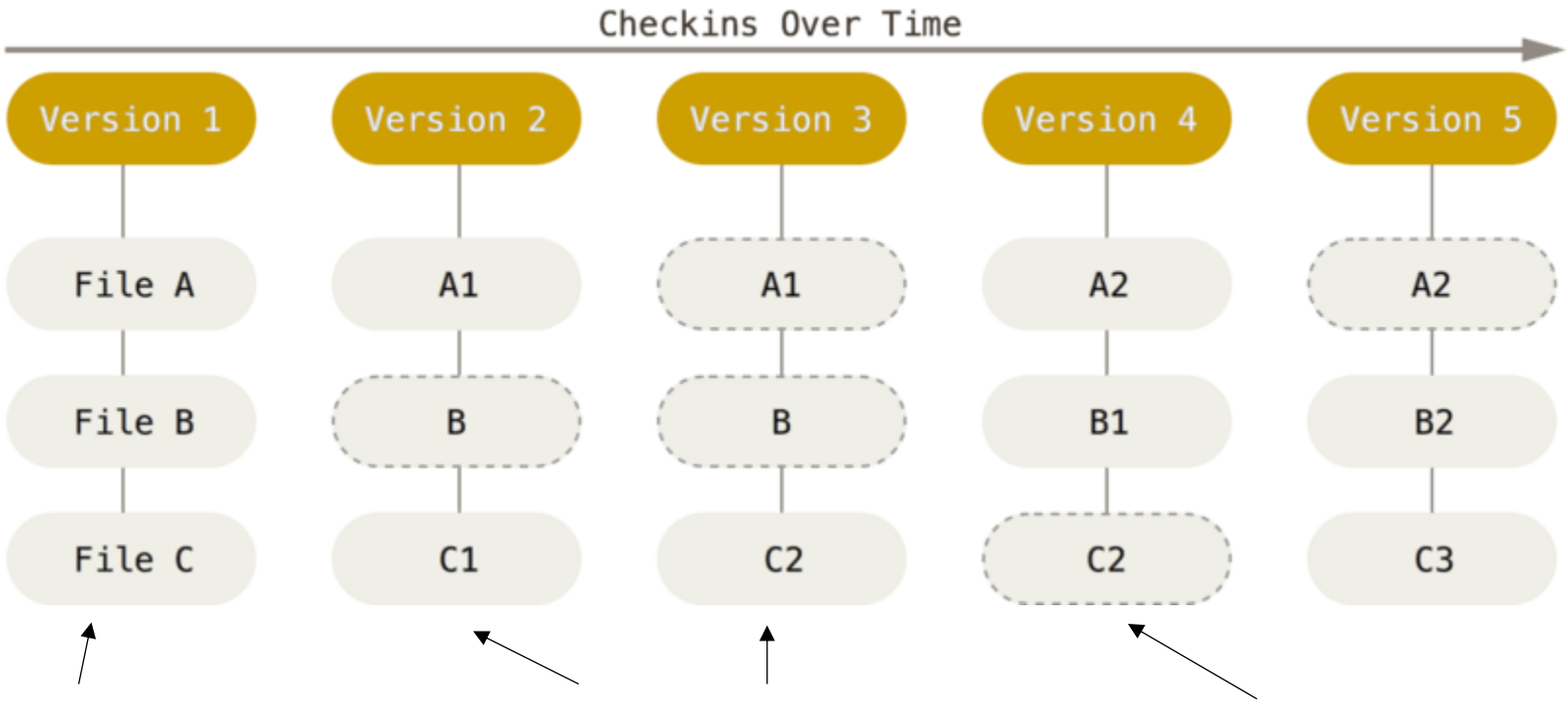


SVN (and many others) start with an initial version of a file

For each change, only the difference (delta) is recorded

To compute, e.g., File C at Version 5, three deltas are applied to File C to obtain the correct state

Git stores sequences of Snapshots



Git stores **snapshots** of each file

Each change leads to a new snapshot

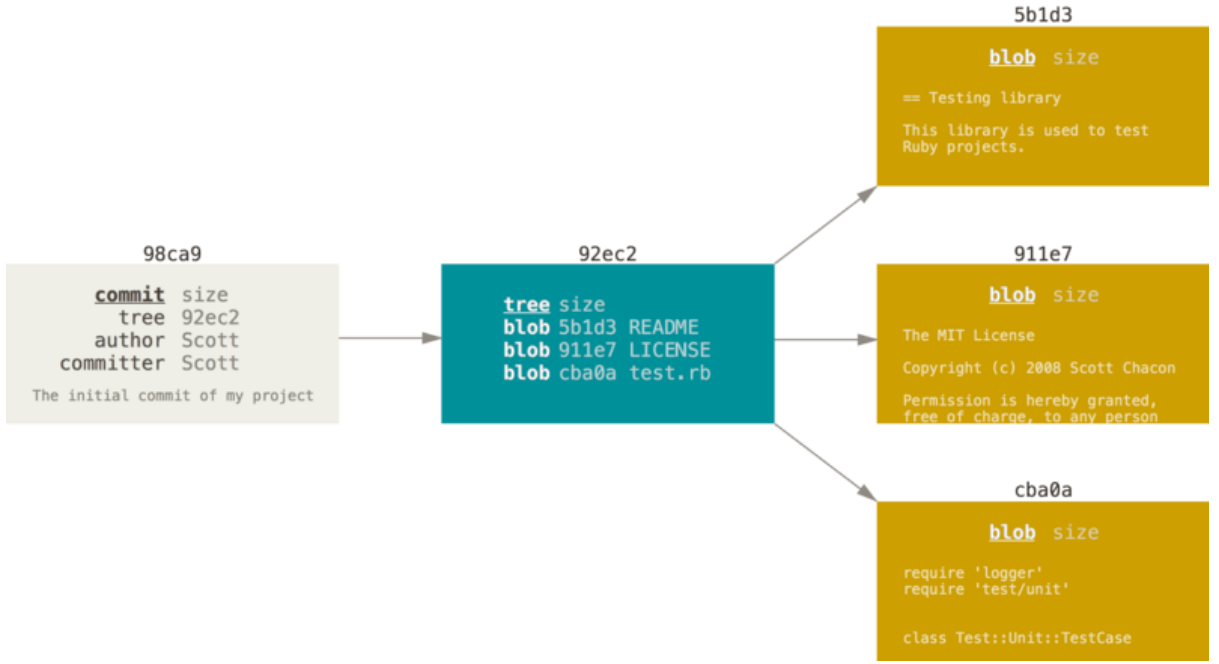
Without a change, the snapshot stays the same

https://git-scm.com/book/en/v2/Getting-Started-What-is-Git

Commits

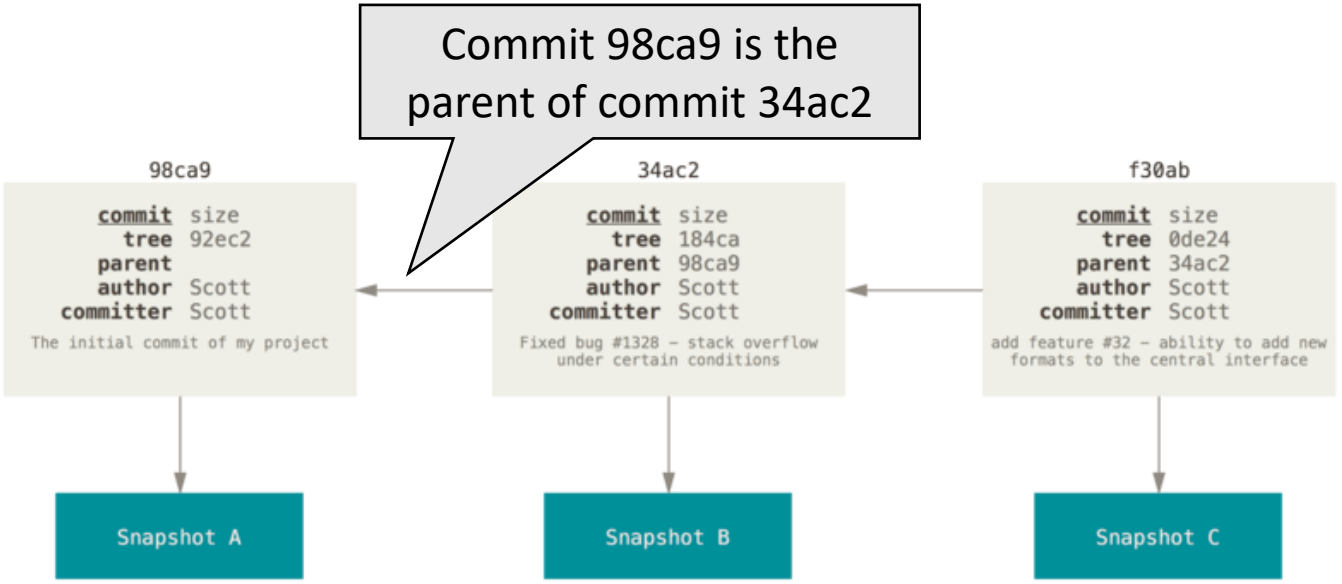
- Changes that are submitted to the version control system

- Each commit has
  - A size, i.e., the lines added and deleted
  - A tree that lists the contents
    - Points to the snapshot of the repository that is stored
  - An author who wrote the code (+email)
  - A committer who added the code to the version control (+email)
  - A message that describes the commit
  - Author date, committer date as timestamps



https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Linear sequences of commits

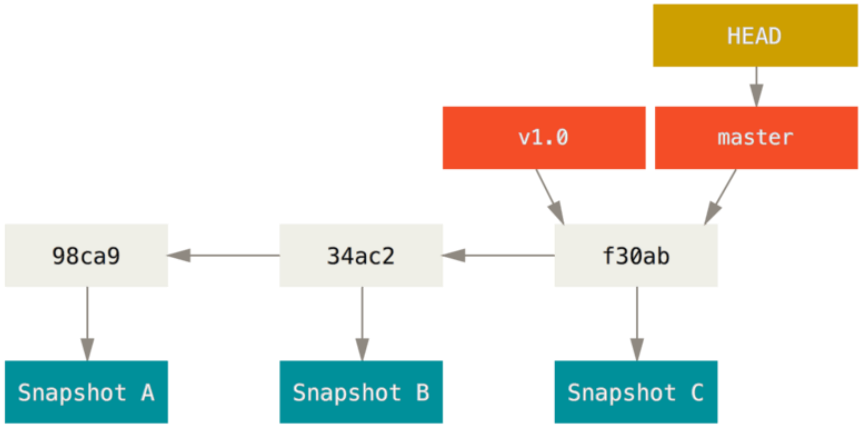


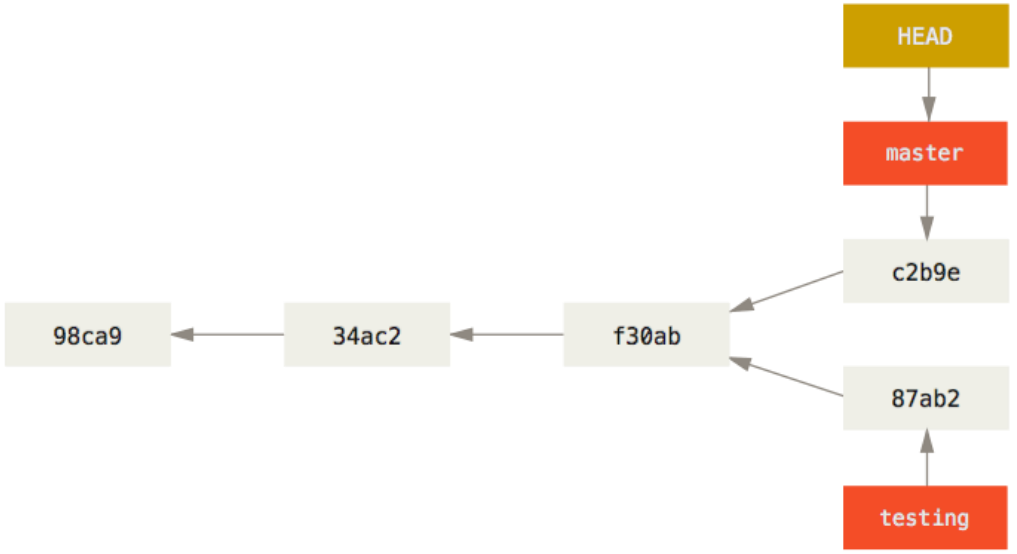
- Subsequent commits are organized as a graph
  - Parent/child relationship between commits
- Each commit is identified by its **revision hash**
  - The revision hash is (usually) calculated as a SHA hash of the snapshot

https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Branches

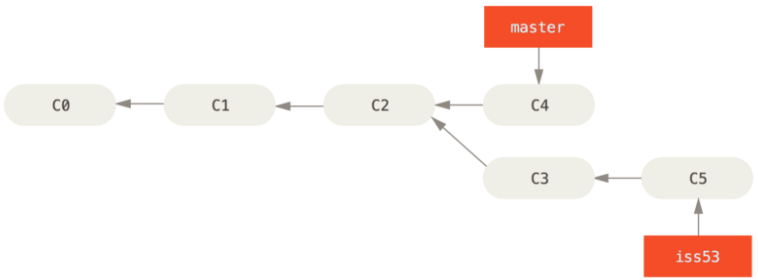
- A branch like v1.0, master (or main) is simply a pointer to a commit
- HEAD points to the branch that is currently being worked on
  - Remote HEAD points to default branch (usually main/master)
  - Local HEAD points to the commit you have currently checked out
- Branches usually used to allow parallel strands of development
  - The example shows two different commits in the master and the testing branch





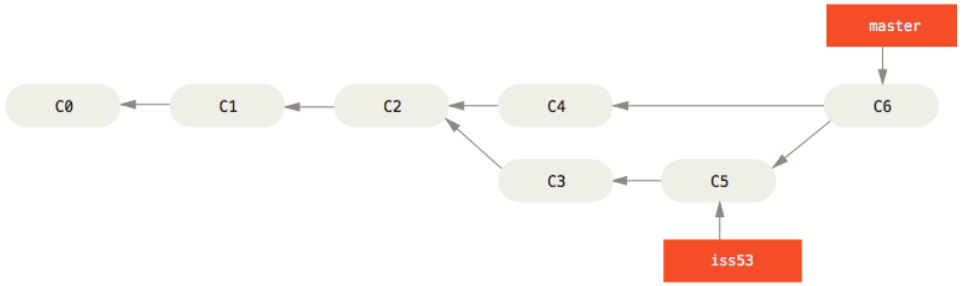
https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

Merge commits



Parallel development on master branch (C4) and iss53 branch (C3, C5)

Merge commit

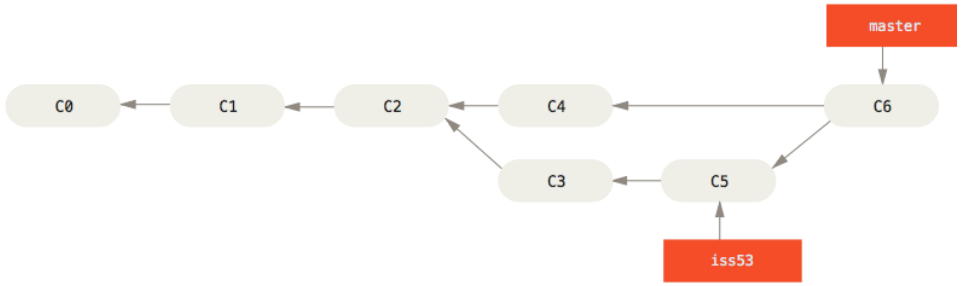


C6 merges branches back together

https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging

The Git graph

- Commits are organized as a Directed Acyclic Graph (DAG)
  - Not strictly enforced. It is possibly to create cycles!
- Commits can also have more than two parents/children
  - … if multiple branches are created after the same commit
  - … if multiple branches are merged with the same merge commit



- Commits without parent are called root commit / orphan commits
  - E.g., the initial commit within a repository
- Repository can have multiple root commits!
  - Allows all sorts of constructs
  - Common on GitHub: homepage of a project in a gh-pages branch that is completely independent of the development branches (i.e., no merges between these branches)
  - Also allows managing many projects in a single repository, each project with the own independent branches

The promises of mining git

- Local developer copies even allow mining of local progress that are never available in central repository
- Commit, branch, and merge information provide rich history
- Git records all information regarding conflict resolution
- Git commits may be signed and even create an official paper trail
- Difference between author and committer allows better attribution of contributions
- Complete repository is locally available
  - No communication with server required for collecting data
- Change tracking allows identification when which line was changed by whom

- Many VCSs have converters to Git
  - E.g., svn2git
  - Beware: may sometimes create strange artifacts or broken conversions like orphan commits (no parents or children)

Bird, Christian, et al. "The promises and perils of mining git." *2009 6th IEEE International Working Conference on Mining Software Repositories*. IEEE, 2009.

The perils of mining Git

- Working with branches is normal behavior
- There is no single mainline!
  - Cannot assume a linear development path
  - Makes analysis of history difficult
    - C1 is committed to branch B1 in March
    - C2 is committed to branch B2 in April
    - B2 is merged to main in May
    - B1 is merged to main in June
    - From the perspective of main, C2 happened before C1 (as in: merged before), from a developer perspective the work on C1 was finished before C2
- Git history can be rewritten
  - e.g., updates of commit messages, squash commits, undoing of changes, cherry picking, …
  - Even allows log pruning to clean up the repository
- Not always possible to know which branch a commit was made on
  - E.g., did the main branch pull a feature branch to integrate the current state of a feature or did the feature branch pull the master branch to get up to date with latest developments?

Bird, Christian, et al. "The promises and perils of mining git." *2009 6th IEEE International Working Conference on Mining Software Repositories*. IEEE, 2009.

Calculating and tracking changes to a file

- Recall: git stores snapshots, not deltas

→The difference between two states needs to be computed somehow

- If a file is renamed, this needs to be calculated!

- Git uses a straightforward heuristic:
  - Rename = delete old file and create new file with the same content
  - → When a file is deleted, git checks if a file with „similar enough“ content is created
  - → If yes, this is considered a rename

Reason for similarity instead of equality for rename heuristic

```java
package de.ugoe.cs.example;

public class Foo {
    public void bar() {
        System.out.println("Hello World!");
    }
}
```



```java
package de.up.cs.examples;

public class Foo {
    public void bar() {
        System.out.println("Hello World!");
    }
}
```

Move class refactoring: move a class from one package to another

- Moves file Foo.java from src/de/ugoe/cs/examples to src/de/up/cs/examples
- Changes first line of Foo.java to reflect new package name

→ No exact match, but high similarity!

Default value is that 50% similarity is required, but can be configured

Textual diffs and their role for VCSs

- We often analyze changes between files
  - E.g., between two commits
- These changes are usually expressed as **textual diffs**
  - Beginning defines where in a file the change is applied
    - @@ -169,11 +284,26 @@
    - Starting from line 169, 11 lines of the first file are shown
    - Starting from line 284, 26 lines of the second file are shown
  - + indicates line added
  - - indicates line removed
- Same two files can have very different diffs!
  - Depends on algorithm!
  - May affect validity of data and analysis
    - e.g., aligning code review comments with mined changes assumes that the same diff algorithm was used
- Always check if algorithm is relevant and manually select suitable algorithm if required!

```
@@ -169,11 +284,36 @@ public class AmqpMessage {
      * @throws IllegalStateException if the message is read only.
      */
     public void setText(String value) throws IllegalStateException {
+        checkReadOnly();
+        AmqpValue body = new AmqpValue(value);
+        getWrappedMessage().setBody(body);
+    }
+
+    //----- Internal implementation ------------------------------------------//
+
+    private void checkReadOnly() throws IllegalStateException {
         if (delivery != null) {
             throw new IllegalStateException("Message is read only.");
         }
+    }

-        AmqpValue body = new AmqpValue(value);
-        getWrappedMessage().setBody(body);
```

(a) `diff` output using *Myers*

```
@@ -169,11 +284,36 @@ public class AmqpMessage {
      * @throws IllegalStateException if the message is read only.
      */
     public void setText(String value) throws IllegalStateException {
-        if (delivery != null) {
-            throw new IllegalStateException("Message is read only.");
-        }
-
+        checkReadOnly();
         AmqpValue body = new AmqpValue(value);
         getWrappedMessage().setBody(body);
     }
+
+    //----- Internal implementation ------------------------------------------//
+
+    private void checkReadOnly() throws IllegalStateException {
+        if (delivery != null) {
+            throw new IllegalStateException("Message is read only.");
+        }
+    }
```

(b) `diff` output using *Histogram*

Nugroho, Y.S., Hata, H. & Matsumoto, K. How different are different diff algorithms in Git?. Empir Software Eng 25, 790–823 (2020)

Code and VCS rich information source

- All aspects related to code can be analyzed
  - Often the complete development history
  - Caveat: analysis tools might have problems with old versions
- Authorship and committer information provides data about underlying social networks
  - E.g., teams working together
- Commit messages are, on their own, a very informative
  - Summary of changes often indicates type of change
  - Sometimes even explains design decision
  - Can contain links to other information
- Timestamps allow a better understanding of who worked when
  - E.g., who contributes during work hours and who contributes on their own time in open-source projects?
- Commit graph allows understanding of development process and how teams work in parallel
  - Who pushes/pulls from where and when? How are branches used?

Issue tracking

Issue tracker

- Allows developers to report and track any issue related to the project

- Issues are a versatile concept that can be used for many purposes
  - Writing down requirements
  - Change requests
  - Bug reports
  - Questions about a system
  - Task lists

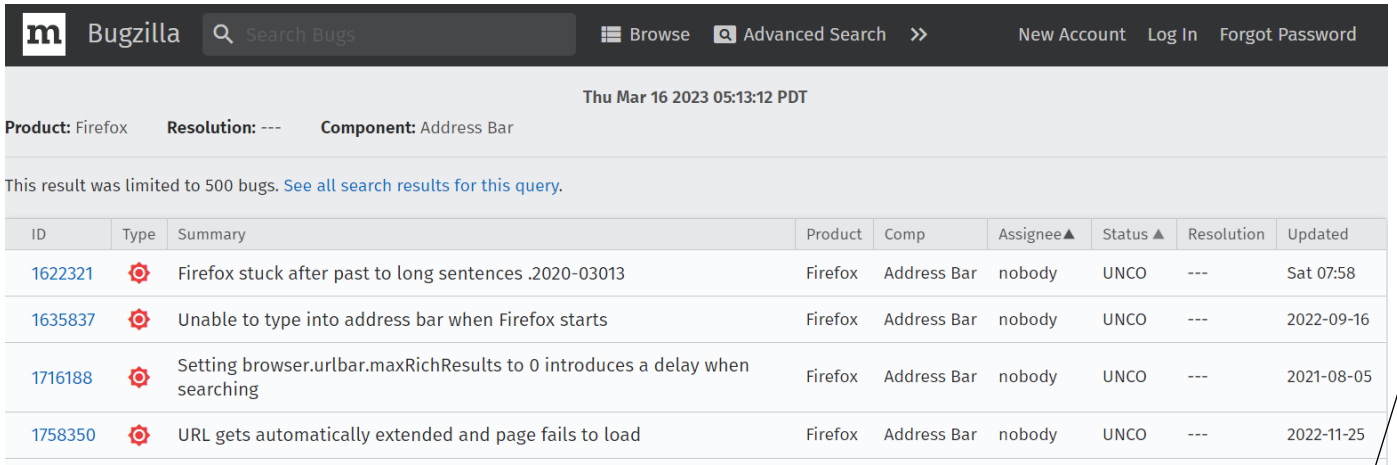
- Different issue trackers have fields to track different aspects of issues
  - Title, description, severity, labels
  - For example, severity, labels, discussions, …

Bugzilla

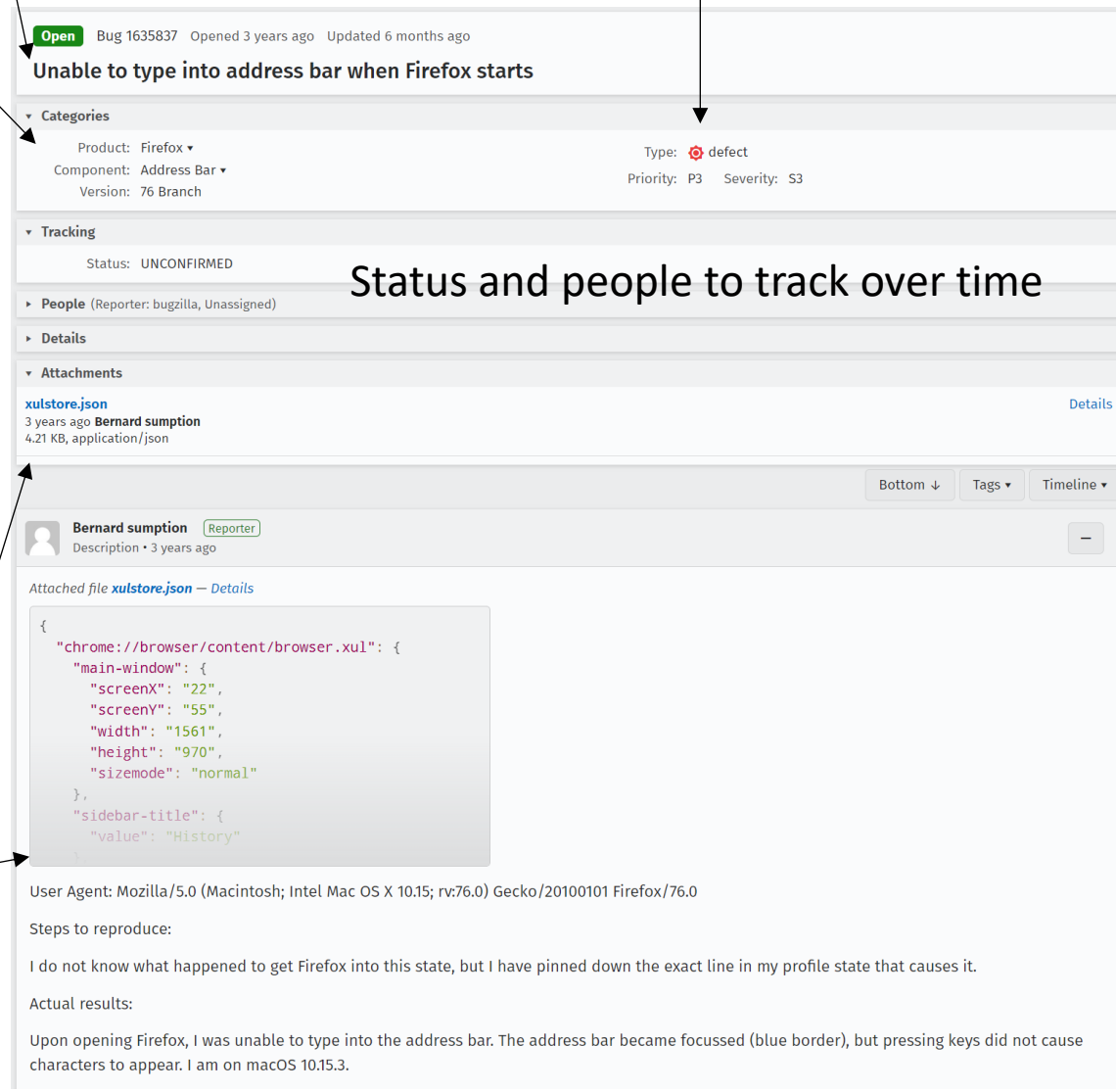
Short summary as title

Issues have a type, e.g., defect, feature request
Priority and severity as metadata to aid planning

Can track different products, components, and versions



ID as number



Status and people to track over time

Attachments provide details, can also contain code (e.g., patches)

Detailed description with markup

Additionally: discussion with free text comments

https://bugzilla.mozilla.org/show_bug.cgi?id=1635837

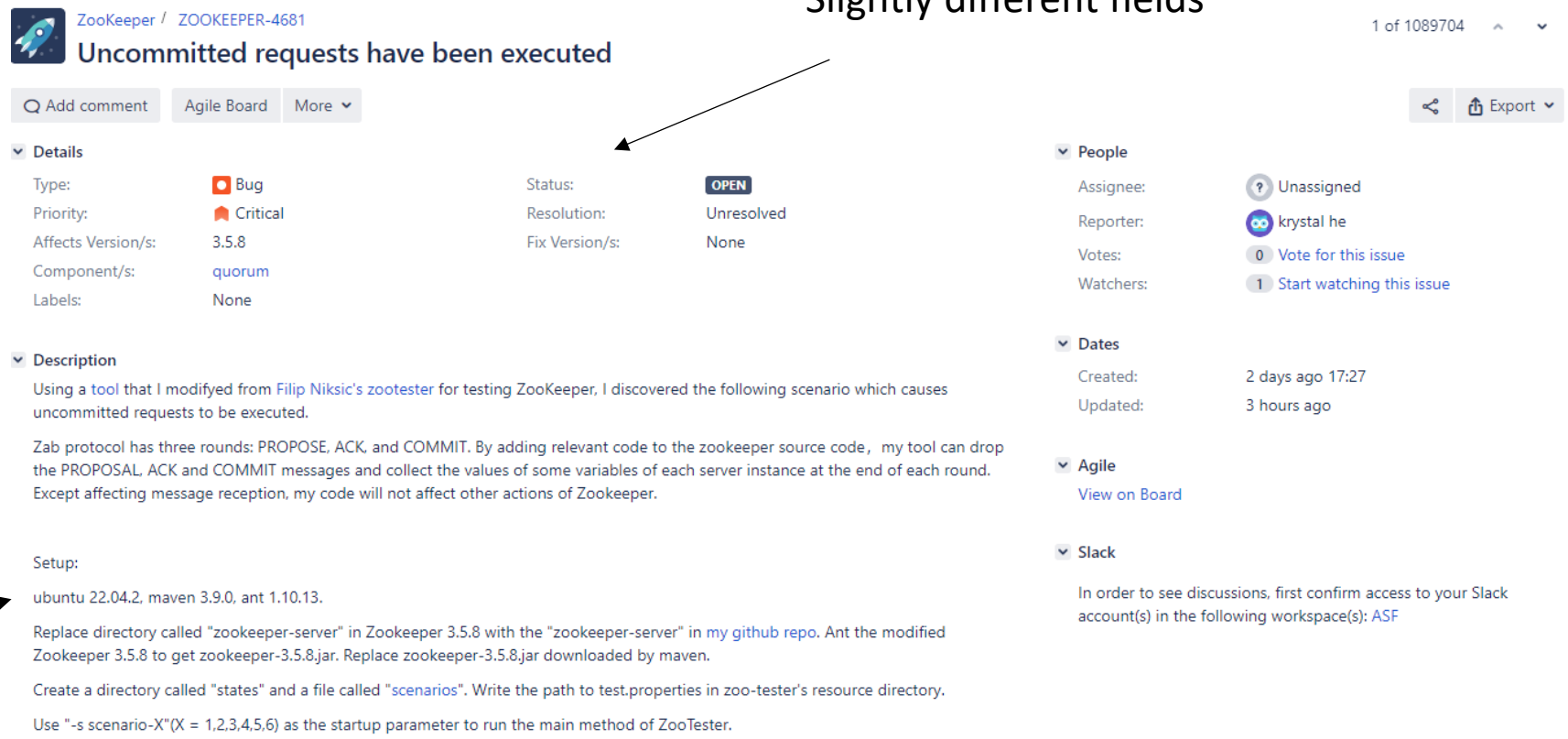
Jira

IDs include project names

Slightly different fields

In general, similar to Bugzilla

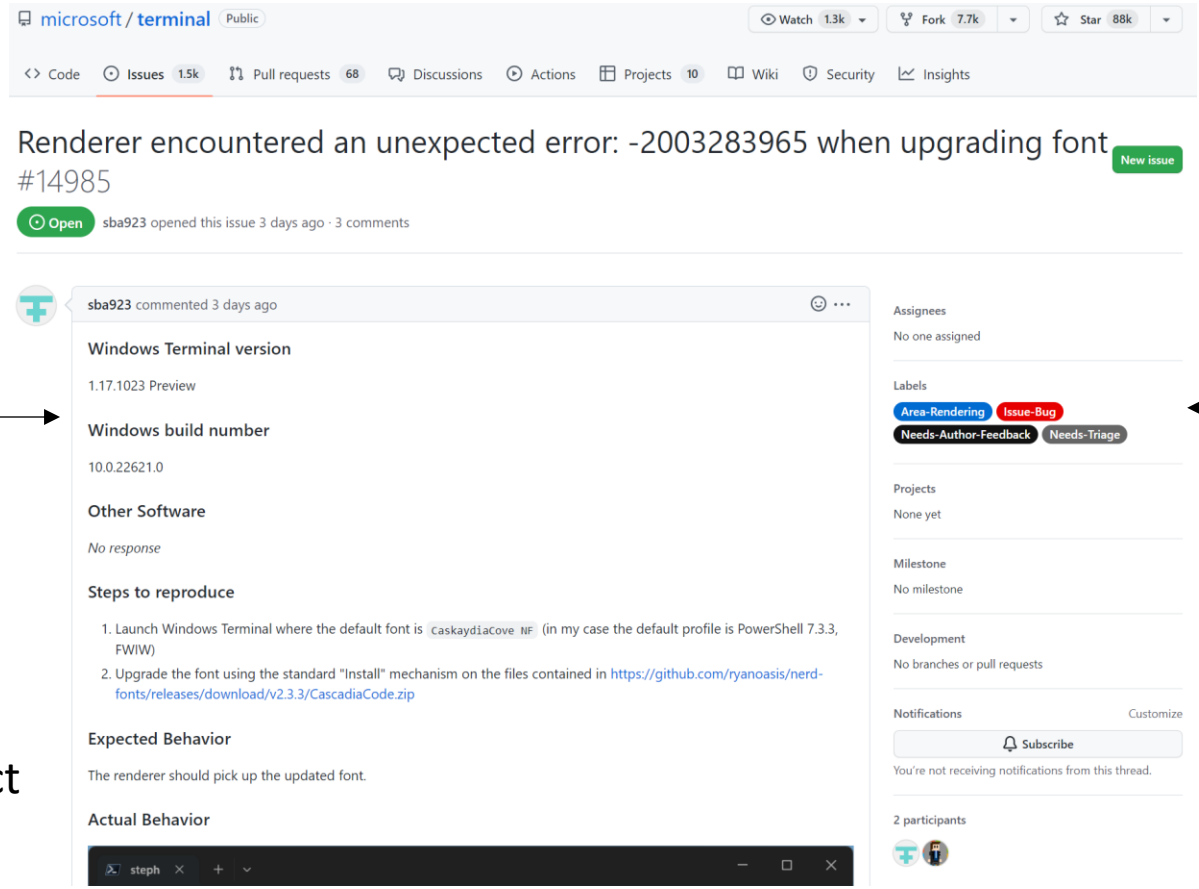
Different markup options



https://issues.apache.org/jira/browse/ZOOKEEPER-4681

GitHub Issues

Description is just the first comment



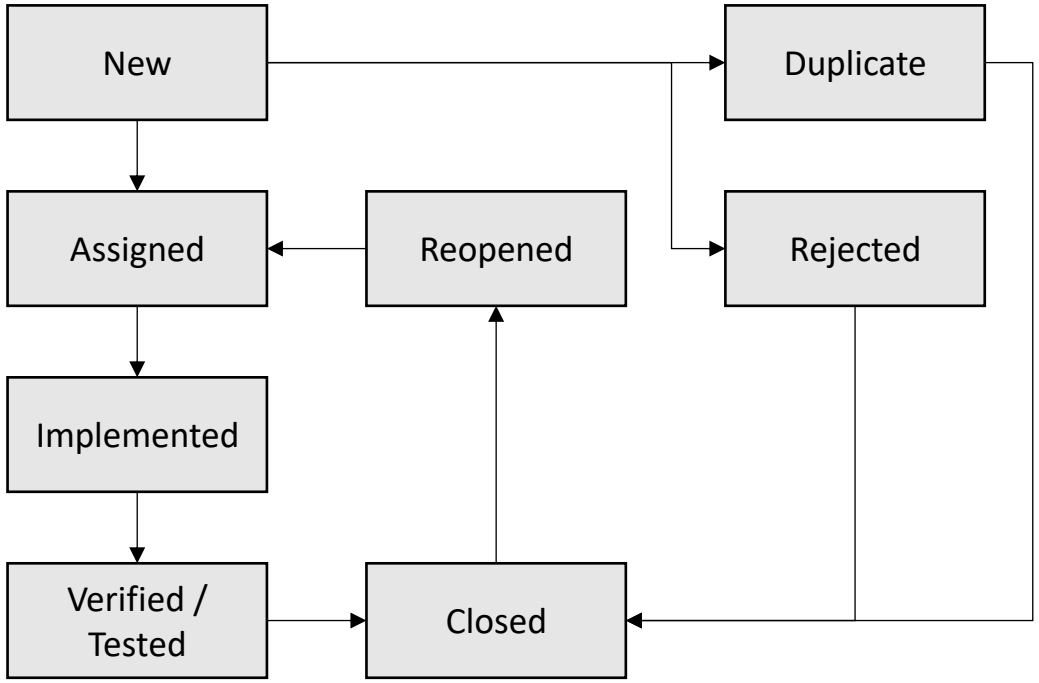
No specialized fields. Instead labels for everything.
→ Analysis of metadata is more difficult

And another markup dialect

https://github.com/microsoft/terminal/issues/14985

Issue life cycle

- Issues go through a whole lifecycle
  - Figure shows just one example. Can also be more complex.
- This lifecycle is usually supported by the tracker
  - Bugzilla/Jira: separate fields in meta data
  - GitHub: Open/closed as status, everything else is only indirectly supported via labels
- All changes to the issue are reported as events
  - Including status changes in the lifecycle and comments



→ Data collection from issue trackers must also capture events

Editing of content as events

- Common assumption: fields stay the same, unless they are for tracking the status
  - E.g., we assume that the description and comments remain unchanged, but that the status is regularly updated
- Not always true
  - The description might be updated to include new information to avoid that this is hidden in the comments
  - Somebody may want to add typos
  - Somebody may want to delete a comment that was unjustified
- We need to track the change history as well
  - Encoded in events
  - Need to make sure with data collection that we can determine the full state of an issue at any time
- Requires decisions when mining
  - Do you store the description as it appears at the time of collection and trace how it was changed to prior states through events?
  - Do you store the description as it was initially created and trace how it was changed to the state at the time of collection through events?

  → Often, the API of the tools support one of the cases natively

Issue tracking = process tracking

- Issue trackers maintain a view on a project from a development process perspective
  - Requirements
  - Changes
  - Bugs
  - Release planning
  - Task assignment
  - Milestones

- Vital data about project health
  - How many issues are open/created/closed at a given time?
  - How does this change over time?
  - What are the types of issues? How severe are they?
  - Who contributes to the discussions and reporting? Core team or outsides as well?

Code reviews

Code review tools

- Code review is an established best practice for software development

- Earlier:
  - Checkout code into local environment, read code there, provide comments separately, e.g., via Email or issue tracker

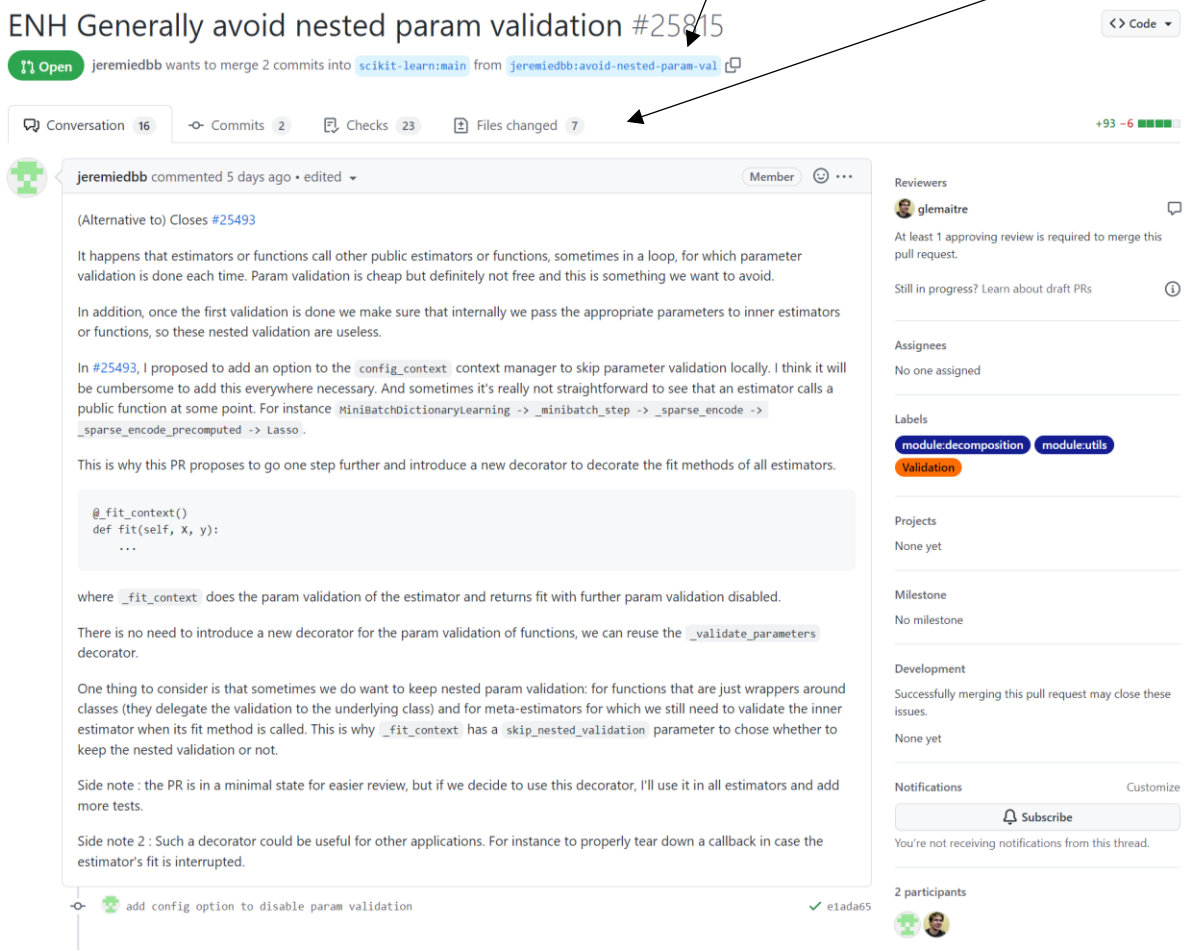
- Nowadays often supported by tools
  - Developer declares code as ready for review
  - Reviewers get to see the code in a dedicated tool
  - Comments and annotation are provided via the tool
  - Changes to the code due to comments are merged back into the tool
  - Once reviewers sign of, code can be merged into main development branch

GitHub pull requests

- Pull requests serve a dual role
  - To initiate a merge commit from a branch
  - To facilitate a review of the suggested merge
  - Opening a pull request does not require rights to commit to a repository
    - Allows anyone to contribute a pull request while maintaining restricted access with respect to write rights to the repository

Shows from which branch should be pulled

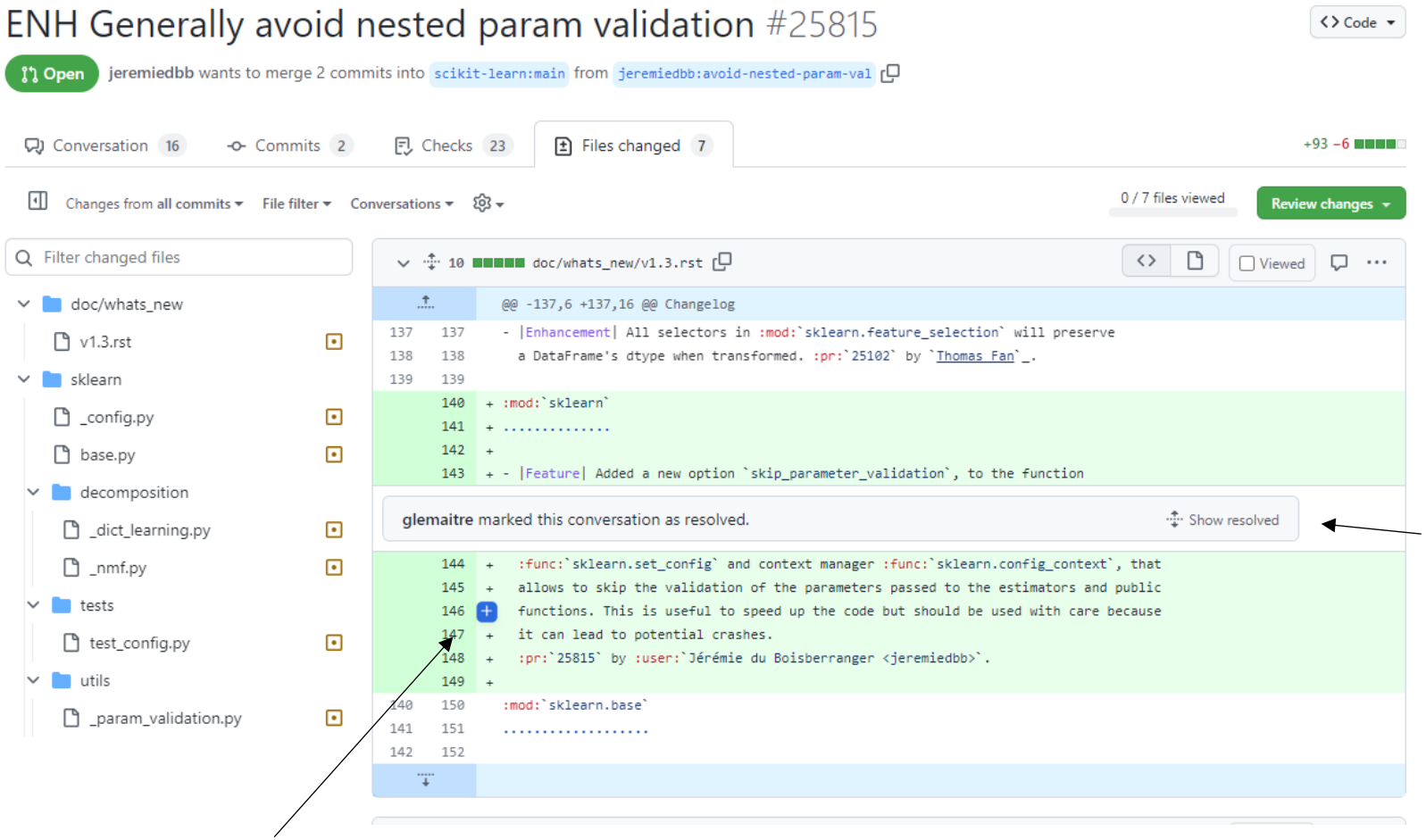
Can view files changed and commits on branch



Landing page similar to GitHub Issues

https://github.com/scikit-learn/scikit-learn/pull/25815

Code review within pull requests



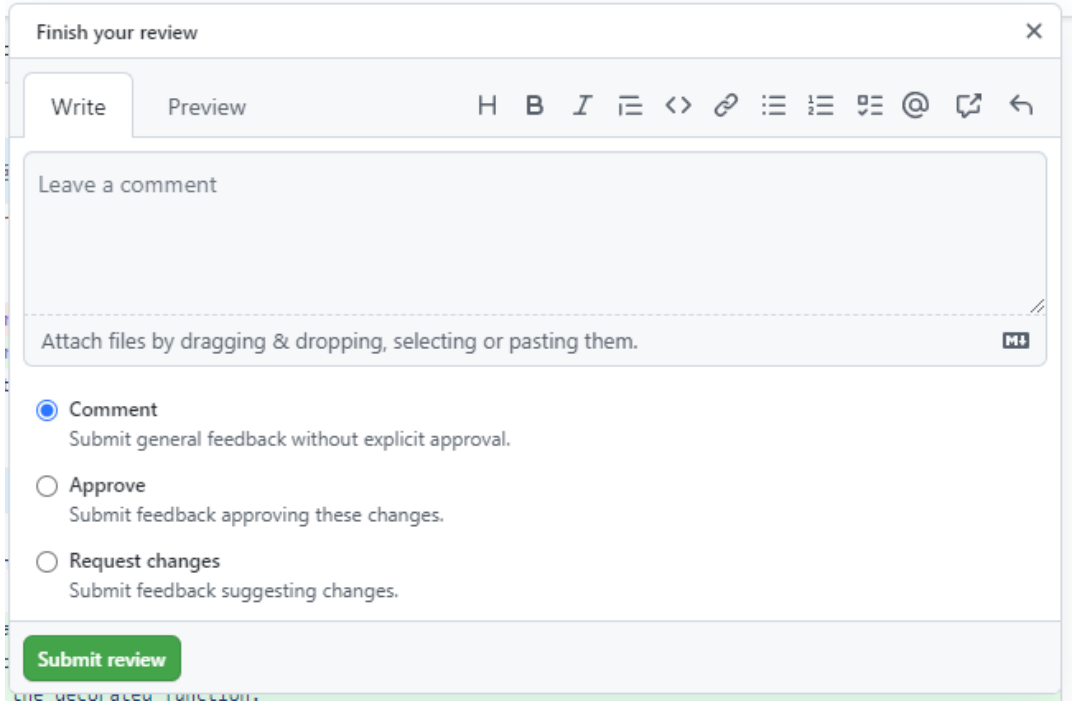
Textual diff of all changes

Can have discussions on individual lines or ranges of lines

Add comment by simply clicking the plus button

https://github.com/scikit-learn/scikit-learn/pull/25815

Overall comment at the end



Summary comment

Suggestion to go forward

Automatically added as comment to the discussion upon submission

Collecting data about pull requests

- The obvious part: data directly from the pull request
  - Issue tracking data from overview (incl. discussion)
  - Comments on individual lines incl. discussion
  - Comments on overall pull requests
  - Approval/comments/rejection ratings

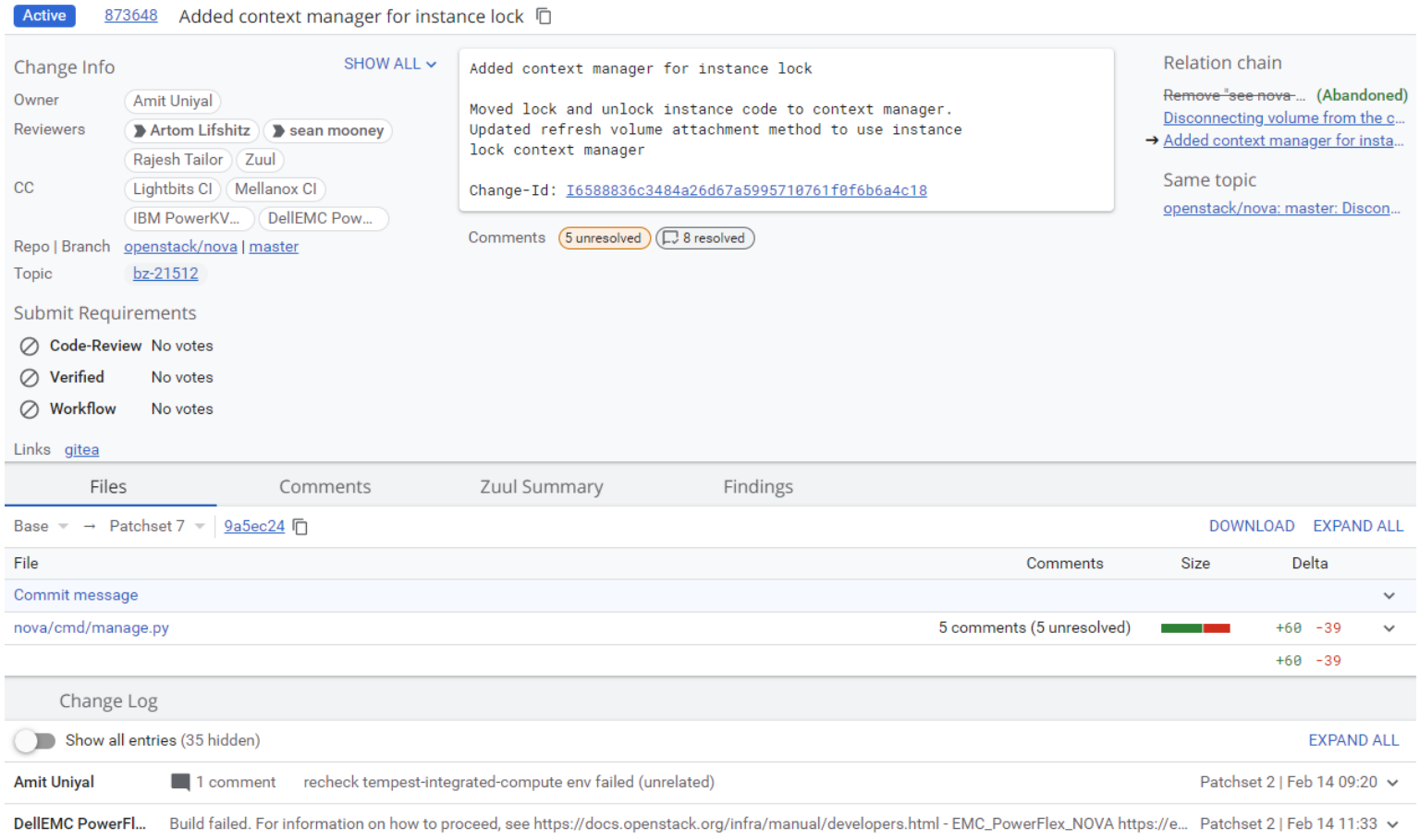
- The tricky part: data about the code change
  - This is actually VCS data
  - If captured, what exactly is collected?
    - Textual diffs?
    - Files that are reviewed before and after change?
    - What about commits that happen during pull request reviews, e.g., to address comments?
  - Requires integration with VCS data collection if the actual code changes should be considered as well!

Gerrit as dedicated code review tool

Similar to review within pull requests

More fields dedicated to specific aspects, similar to difference between GitHub Issues and Jira/Bugzilla

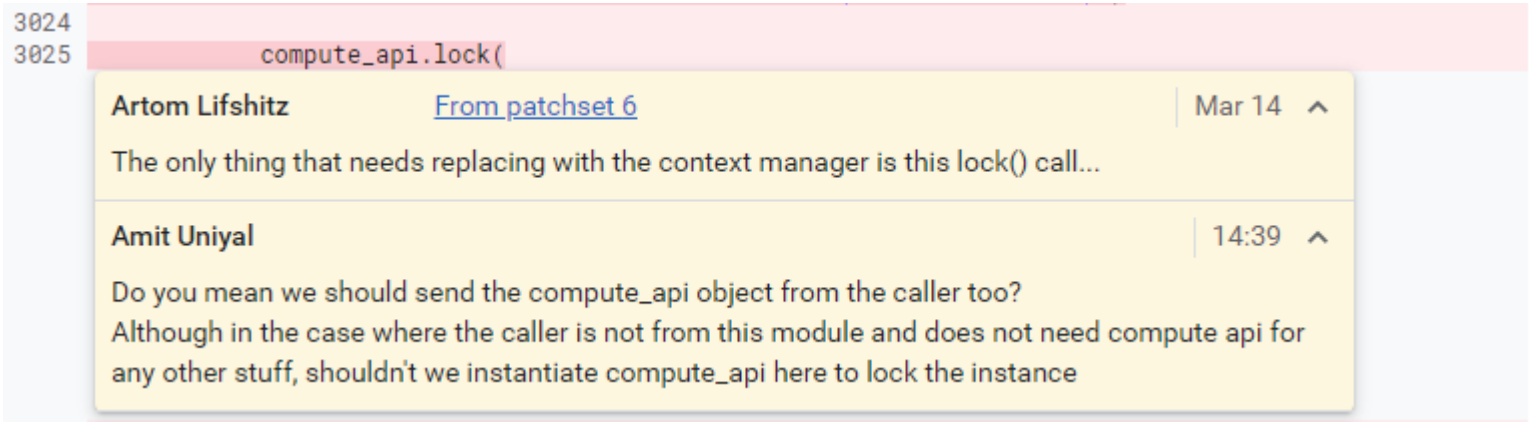
Can be both directly integrated with VCS system or be just for the patches



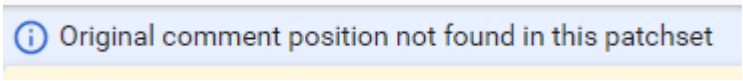
https://review.opendev.org/c/openstack/nova/+/873648

Commenting within Gerrit

Comments can be added to any line, incl. discussion



It‘s possible that comments have no location anymore, because the related part was removed during the review



Collecting from dedicated tools like Gerrit

- Similar to pull requests, but some notable differences

- Possibly independent of VCS system
  - Means that patches are provided directly to the review system
  - Tracking of patches required, cannot be offloaded to parallel mining of VCS system
  - Relating review information back to VCS system requires more effort

- Because of pure patch-based approach, comments can lose their location
  - Hard to handle in review tool
  - Unclear how to handle during data collection
    - If history of review is included, comments can be associated with the original location
  - Might be difficult to identify if locations are lost
    - The placement of comments in a view is done by the tool
    - If comment cannot be placed anymore, because part is gone, this must be detected by the data collector!

Continuous integration systems

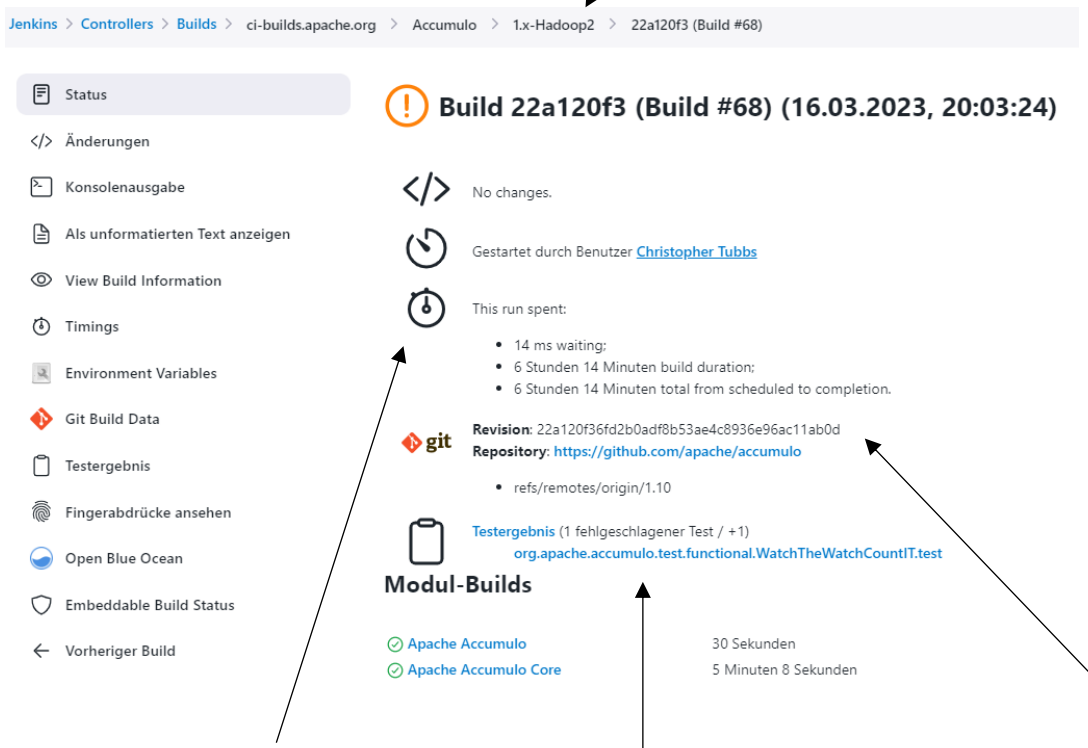
Continuous integration tools

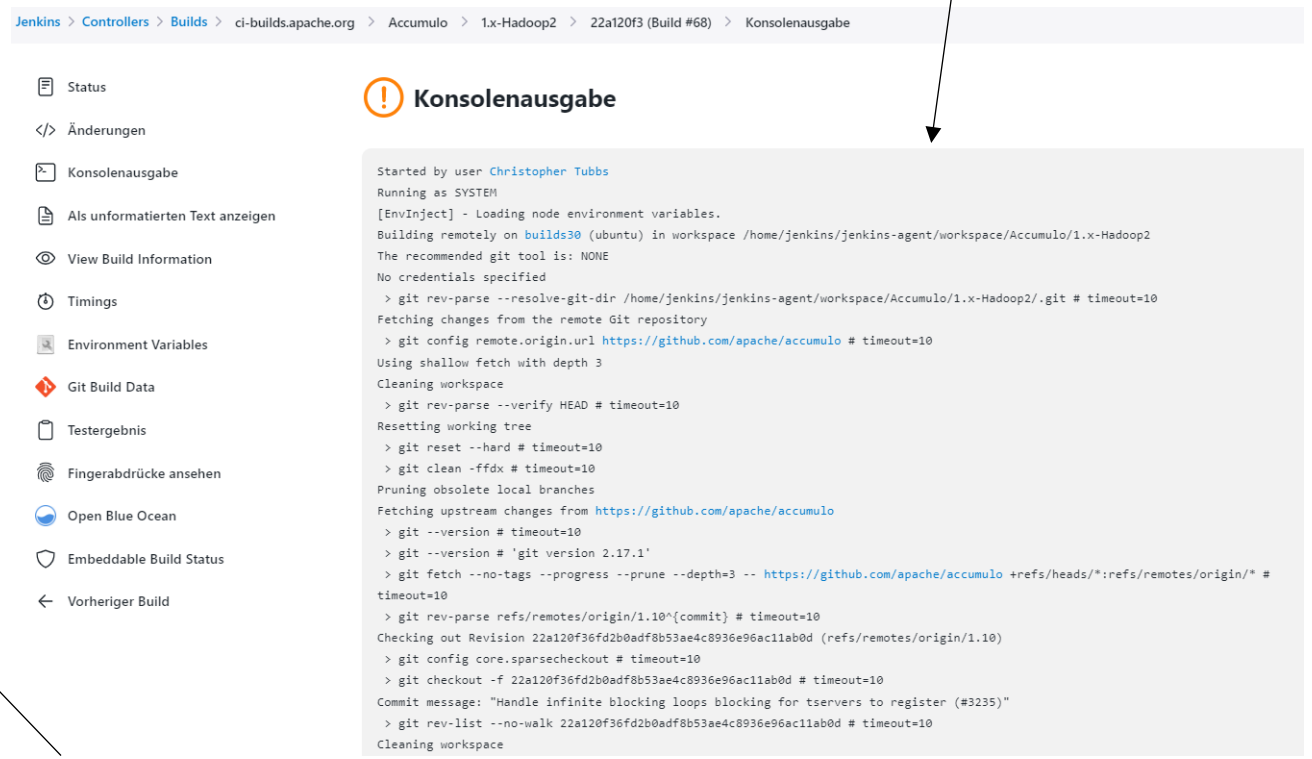
- Tools that automatically build software when triggered
  - Regular builds, e.g., nightly builds
  - Builds for every commit and/or pull request
- Often multiple builds to tests software in different environments, e.g., operating systems
- Often go beyond building the software and also run tests, create and publish reports
- Often used as part of a DevOps pipeline to facilitate the automated deployment of artifacts (binaries, documentation) upon successful builds
  - Variant: deployment only when a tag or similar is created

Jenkins

Environment

Full log of build available





Metadata about build

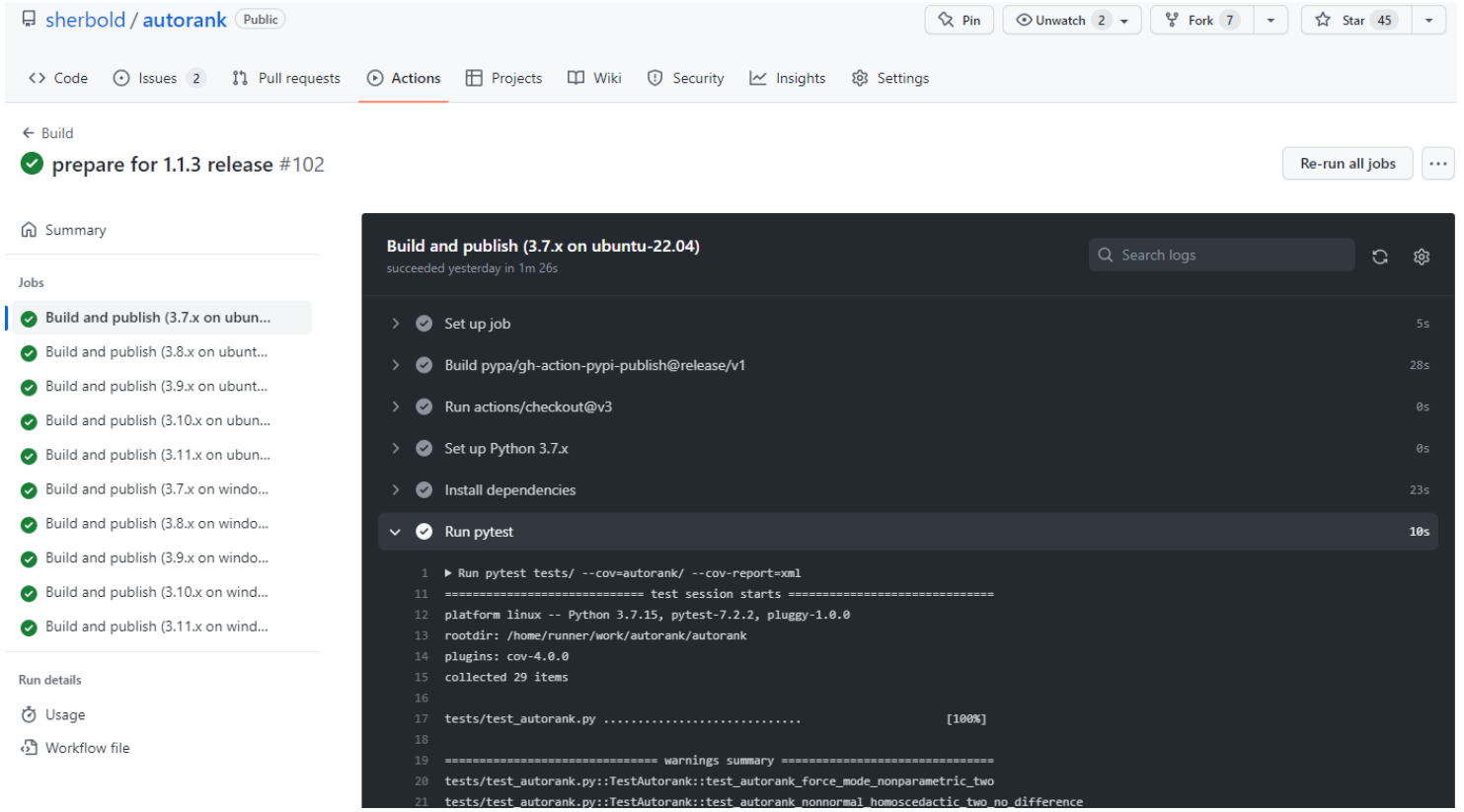
Test results are recorded

Version that was build

https://builds.apache.org/job/Accumulo/job/1.x-Hadoop2/68/

GitHub Actions

- Build capabilities similar to Jenkins
- No reporting beyond build status
  - Information needs to be extracted from build logs instead
- Nowadays very popular because it is easy to use from GitHub
- GitLab Runners provide a similar concept
- External offers with similar functions as well, e.g. Travis CI and Circle CI



https://github.com/sherbold/autorank/actions/runs/4429355993

CI data = build data

- Recall: building software retrospectively is hard!
- CI data provides a remedy: we still cannot execute the software ourselves, but we can see old build results
  - Often includes valuable data, e.g., about tests
  - Depending on the tool (both build and CI), even coverage information and static analysis warning are available
- Raw data in form of build logs requires subsequent parsing
  - Difficult for downstream analysis, often very heterogeneous
    - Between projects
    - Between build tools
    - Between CI systems
    - Between programming languages
    - …
  - Metadata about builds help, but availability depends on CI system
- Data sometimes only retained for a certain timeframe
  - Build logs can be large, which requires cleanups

Telemetry data

Telemetry data about software

- Software produces lots of telemetry data during operation
  - Usually, some form of log data collected

- Error logs
- Web logs
- Logging output from applications
  - Often structured, with different levels for logging (e.g., info, debug, warning, critical)

- Sometimes collected locally, sometimes transmitted to central servers

- Sometimes the only way to observe system behavior
  - E.g., with distributed systems like Apache Spark, where execution logs do not necessary appear locally at all, but rather in a specialized history server

Collecting telemetry data

- No single approach, depends completely on technology!

- Common variants
  - Collect text files from local file systems
    - E.g., from developers, servers
  - Logging server with API from which logs can be retrieved
    - E.g., history server
  - When data collection is not retroactive but live: integrate additional end-point for mining in logging infrastructure

Artifact repositories

Artifacts repositories and dependencies

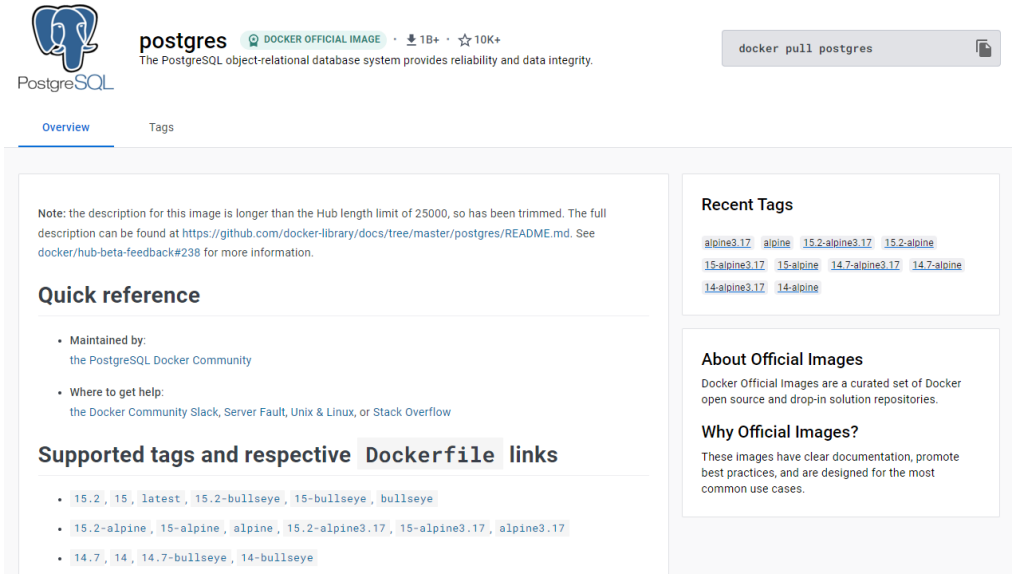
- Dependencies often not (anymore) bundled directly with code

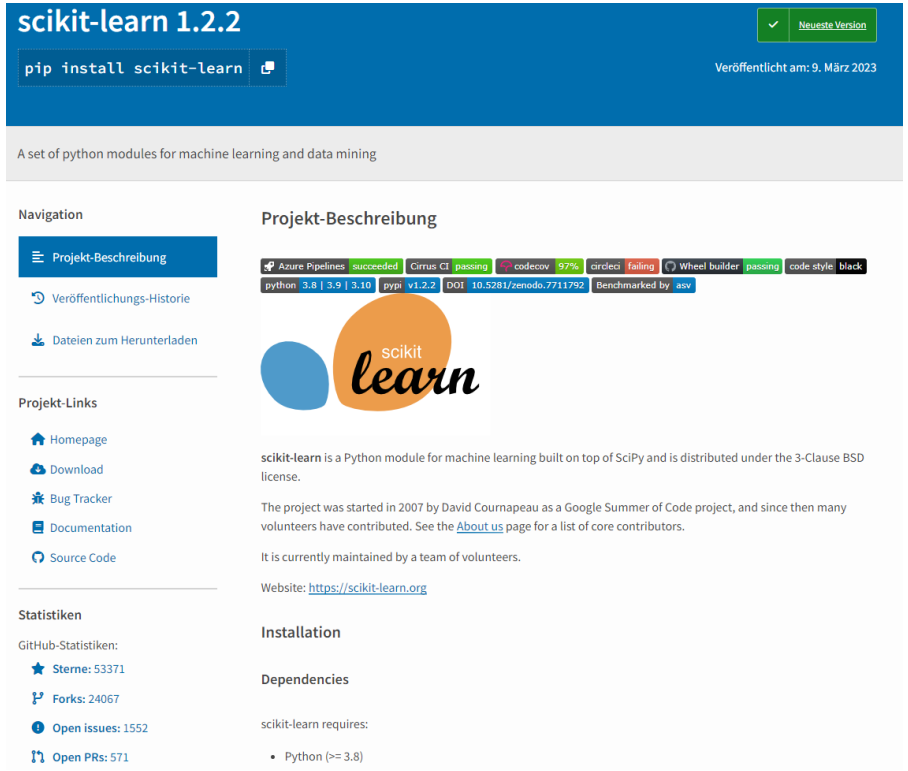
- Instead:
  - Define dependencies in a build or configuration file
  - Retrieve them on demand

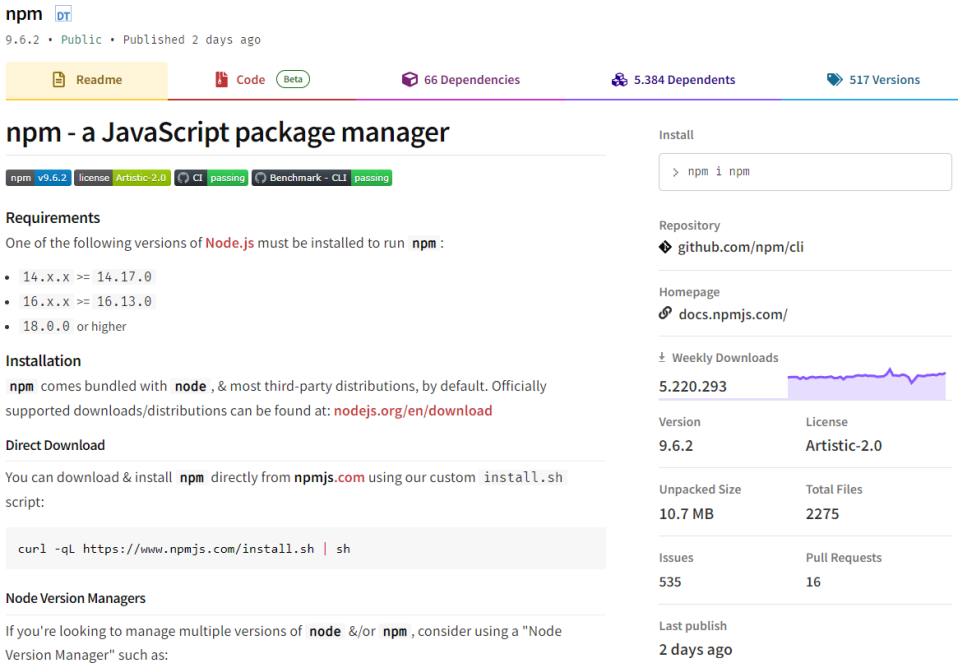
- Different repositories for different parts of the lifecycle
  - Repositories with libraries like PyPi, Maven Central, or npm to retrieve them for development and compilation
  - Pre-configured VM or container images, e.g., from Dockerhub to setup execution environments
  - Full application binaries incl. configuration files for end-users, e.g., with tools like apt-get or chocolatey that can access multiple repositories

Different tools, similar data

- Binaries, version, documentation
- Usually also define dependencies with references to other artifacts in the same repository







Perspective on binaries and relationships

- View on dependencies beyond build and configuration files
- Analysis may be combined with insights into usage
- Can analyze binaries without having to compile source code
  - Way to circumvent problems with retrospective compilation of old code
- Opportunities also depend a lot on programming language
  - E.g., many Java static analysis tools work with bytecode anyways
- Recent supply chain issues show relevance of such repositories for analysis

Forges

Collaborative development

- Combines many (or even all) of the already discussed tooling into a single platform
- GitHub is currently the most popular example with over 100 million registered developers
- Many others as well
  - GitLab and GitBucket as platform similar to GitHub with open-source code
  - Launchpad as popular alternative in the FOSS world maintained by Canonical, the makers of Ubuntu
  - SourceForge as very successful forge in the 2000‘s, when mainly SVN was used
  - FusionForge and GForge as open alternatives to SourceForge with self-hosting
  - …
- Exact features depend
- Success depends on both features and user base

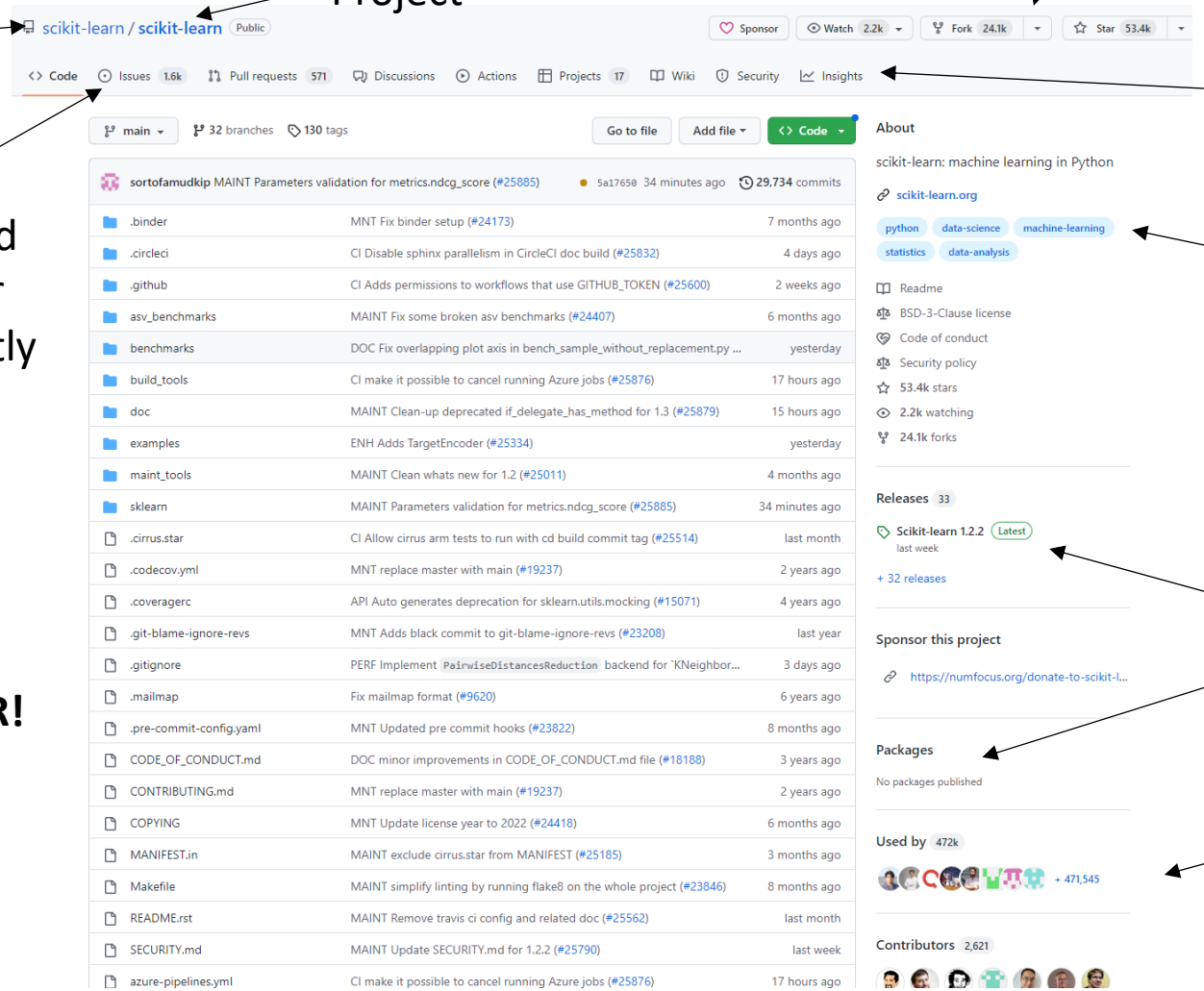
GitHub as example

Organization

Project

Data about popularity

Systems we discussed earlier in the chapter are sometimes directly integrated



Metadata about project, e.g., license, tags, and homepage

Can host binary as package or as releases

Social data about users and contributors

As data source ideal for MSR!

https://github.com/scikit-learn/scikit-learn

Data about ecosystems

- Forges offer more than their individual data sources for projects

- Allows collection of data about whole ecosystems
  - Contributor data
  - Mining who contributes to which projects
  - Structure of organizations
  - Relationship between projects
  - Usage of forks
  - Popularity of with measured through stars

- Combination of good APIs and single source with rich information means that it is easy to study large amounts of data about software development

Forges and inner-source development

- Community aspect of forges comes from the open-source world
  - Public forges from the internet drew open-source developers
- Collaboration between developers of different teams also desirable for closed-source development within a company
- Inner-source development as emerging approach to enable this
  - Host a forge like GitHub Enterprise, GitLab for Enterprise, or similar exclusively for the company
  - Opens code of all projects to all teams and allows easier contributions

→ Enables studies within companies

Communication tools

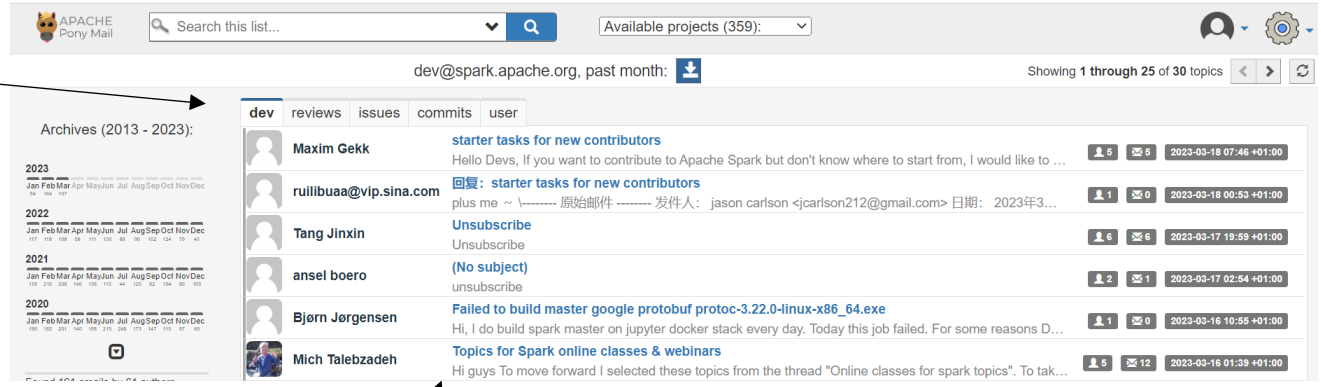
Emails and Chats

- Developers often work in a distributed setting
  - Different teams within a company
  - Different time zones
- May have never met in presence
  - Happens often in the open-source context
- Nevertheless: Need to communicate and discuss
  - Strategy and direction
  - Policy issues
  - Feedback outside of code reviews
- Common solution:
  - Use a mailing list, chat program, or similar

Email Archives

- Fairly simple structure: Email threads with responses

Multiple mailing lists



Email thread incl. subject, initial message, and responses

https://lists.apache.org/list.html?dev@spark.apache.org

Internet Relay Chat (IRC)

Timestamp

Username. Can be changed any time and is (often) not reserved

[02:33] <lubot> [telegram] <lynorian> I don't have a kubuntu install or vm installed but does anyone have the new home screen loading on discover on lunar
[02:33] <lubot> [telegram] <teward001> i can try in a bit, i'm downloading Lubuntu Lunar as part of my testbed for some tools I need to test for backports team anyways in Lunar (and i'm a lazy ass so i want to use a GUI XD)
[02:34] <lubot> [telegram] <lynorian> eh I may try kubuntu with this to see if it is working there when making a bug report as that seems useful info but don't actually have an install
[02:44] <lubot> [telegram] <teward001> looks to me like it just sits on "Loading" for the Home tab of Discover (re @lynorian: I don't have a kubuntu install or vm installed but does anyone have the new home screen loading on discover on lunar)
[02:50] <lubot> [telegram] <lynorian> same in both my bare metal machine and my vm
[02:54] <lubot> [telegram] <teward001> ye mine's in KVM so :P
[03:09] <lubot> [telegram] <lynorian> https://bugs.launchpad.net/ubuntu/+source/plasma-discover/+bug/2008968 filed a bug about it
[03:09] -ubottu:#lubuntu-devel- Launchpad bug 2008968 in plasma-discover (Ubuntu) "discover home never loads" [Undecided, New]
[03:35] <arraybolt3> I just noticed that Lubuntu's upgrader no longer has byte counts during downloads of packages in Lunar. Is this a regression ,or was the byte count feature removed?
[03:35] <arraybolt3> (Personally I don't care much about it being gone, but if it's a regression we should probably fix that.)
[05:42] <guiverc> @lynorian, I think I saw a kubuntu user report that recently (this morning? last night? my local time anyway) but sorry I didn't pay much attention as was busy on other things (re: plasma discover... but it's failing to load for me here too it appears)
[05:50] <lubot> [telegram] <RikMills> cant you try reinstalling the previous version? (re @lynorian: https://bugs.launchpad.net/ubuntu/+source/plasma-discover/+bug/2008968 filed a bug about it)
[05:50] -ubottu:#lubuntu-devel- Launchpad bug 2008968 in plasma-discover (Ubuntu) "discover home never loads" [Undecided, Confirmed]
[05:51] <lubot> [telegram] <RikMills> i.e. the 5.27.1 debs from: https://launchpad.net/ubuntu/+source/plasma-discover/5.27.1-0ubuntu1/+build/25621275
[06:11] <guiverc> @RikMills, if I `sudo apt install ./plasma-discover_5.27.1-0ubuntu1_amd64.deb ./plasma-discover-common_5.27.1-0ubuntu1_all.deb` then i get normal actions from plasma-discover

Chat messages

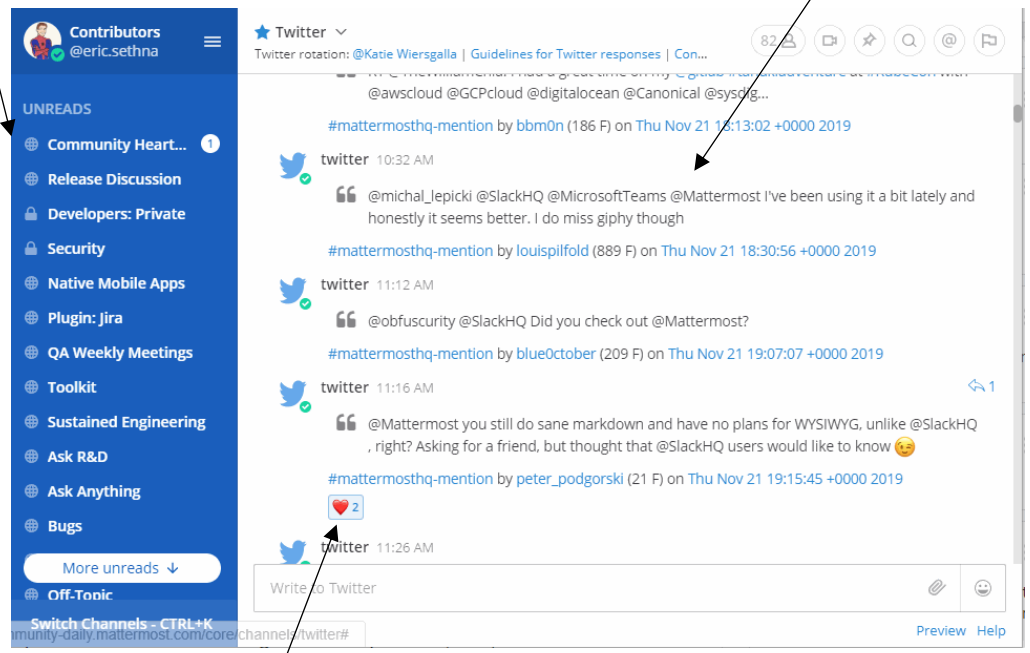
https://irclogs.ubuntu.com/2023/03/02/%23lubuntu-devel.txt

Mattermost and Slack

- Tools for structuring communications

Different channels

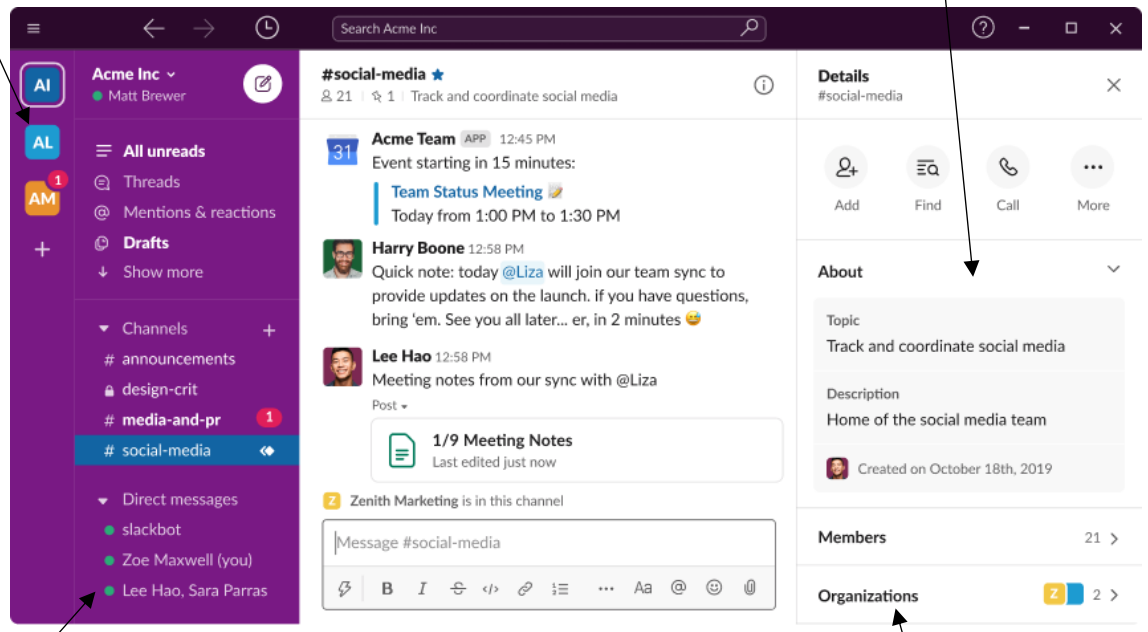
Markups



Reactions

Different workspaces

Metadata about channels



Private messages

Organization structures

https://1435268659-files.gitbook.io/~/files/v0/b/gitbook-legacy-files/o/assets%2F-LorQDNCmgMboJfZ7MPT%2Fsync%2F80eee7a593e13c3272724d2c6e146af9633e2a3a.png?generation=1620935751174679&alt=media
https://a.slack-edge.com/c5da202/marketing/img/downloads/refresned/slack-client-windows-desktop@2x.png

Simple and complex structures

- Mining chats and email conversions themselves fairly straightforward
  - Timestamps, messages, possibly recipients
- Can become very complex
  - Tracking of name changes in IRC
  - Correct identification of who responded to whom with which emails already read in an Email thread
  - All sorts of meta data in tools like Mattermost and Slack
- Tools without login require identify matching when processing data, otherwise analysis may go wrong, e.g.,
  - herbold@cs.uni-goettingen , steffen.herbold@kit.edu, steffen.herbold@tu-clausthal.de, steffen.herbold@uni-passau.de are all the same person!

Social structures and decision making

- Communication data helps us understand social aspects
  - Communication can be modelled as social networks
  - Data about team formation
  - Data about communication hubs and people central to the information flow

- Communication archives provide insights into decision making
  - Especially important for aspects that do not consider a single issue but a broader strategy, e.g.,
    - architecture and design
    - policies for making decisions
  - Helps us to understand not only outcomes, but also the path that led to the outcome

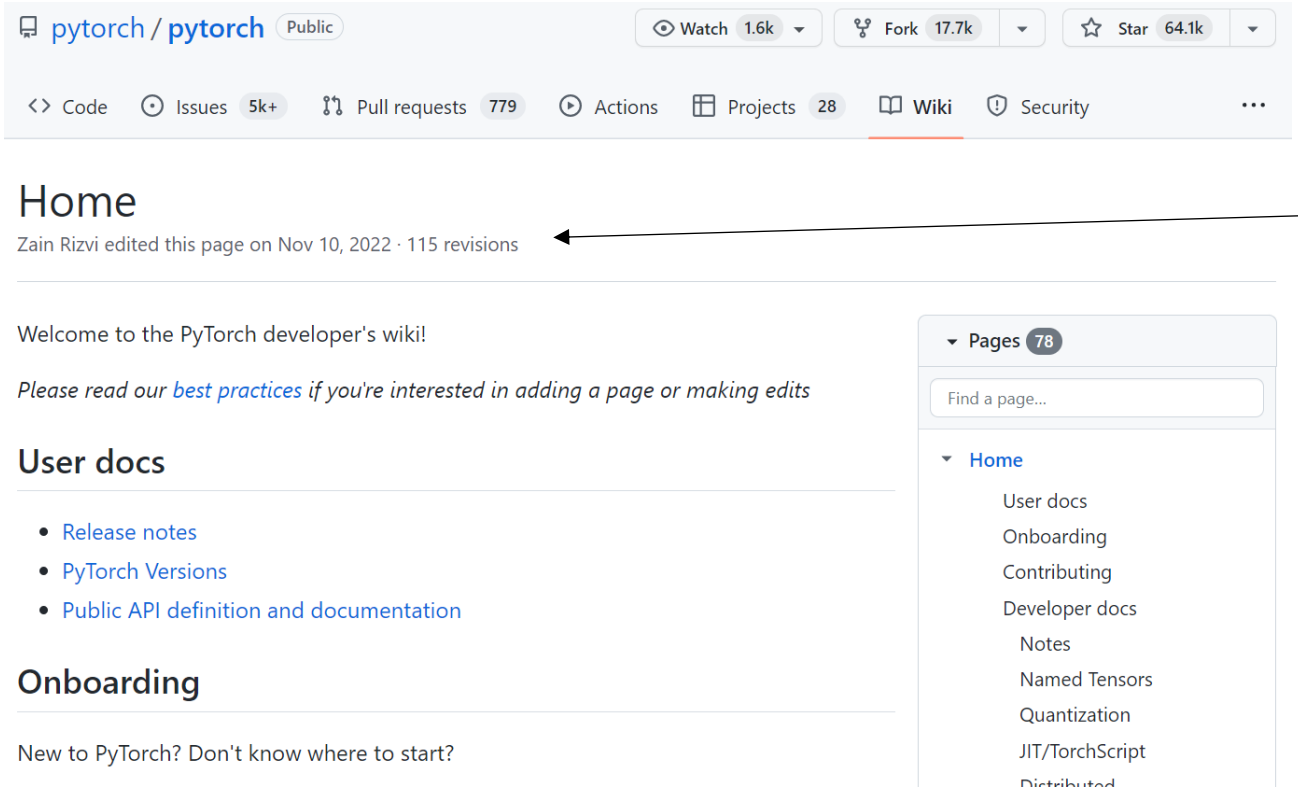
Wikis

Wikis for documentation

- Wikis provide an easy and flexible way to maintain Web pages
- Can be edited collaboratively
  - Easy way for projects to involve users into maintaining content
- Especially popular for documentation about policies design
  - Contribution guidelines
    - Code, issues, reviews, …
  - FAQs for building the software
  - Architecture documentation
  - Developer tools that are used

GitHub Wikis

Detailed authorship information
(on normal Web pages often missing)



History of changes

https://github.com/pytorch/pytorch/wiki

Evolution beyond code

- Wikis enable the study of how documentation, architectures, policies, etc. evolved over time
- Similar to what we can get from other data sources
  - Similar to communication tools, but focused on result, not discussion
- Some Wiki platforms also enable discussion of contents
  - Combines Wiki with communication tool aspects

Q&A sites and developer forums

Discussions beyond project boundary

- Software developers discuss and share knowledge beyond project boundaries all the time

- Developer forums are places to discuss things
  - New releases of software
  - Interesting news articles
  - Sometimes also latest research
  - Advantages and drawbacks of certain technologies
    - You can start a heated discussion by just asking which code editor is best

- Q&A sites like Stack Overflow/Stack Exchange and others help create a public knowledge base
  - Moves effort from looking up knowledge to asking after knowledge
  - Requires others to contribute their knowledge

Stack Overflow

Title that summarizes the question

Votes for/against this question

Body with detailed long-form description of the question

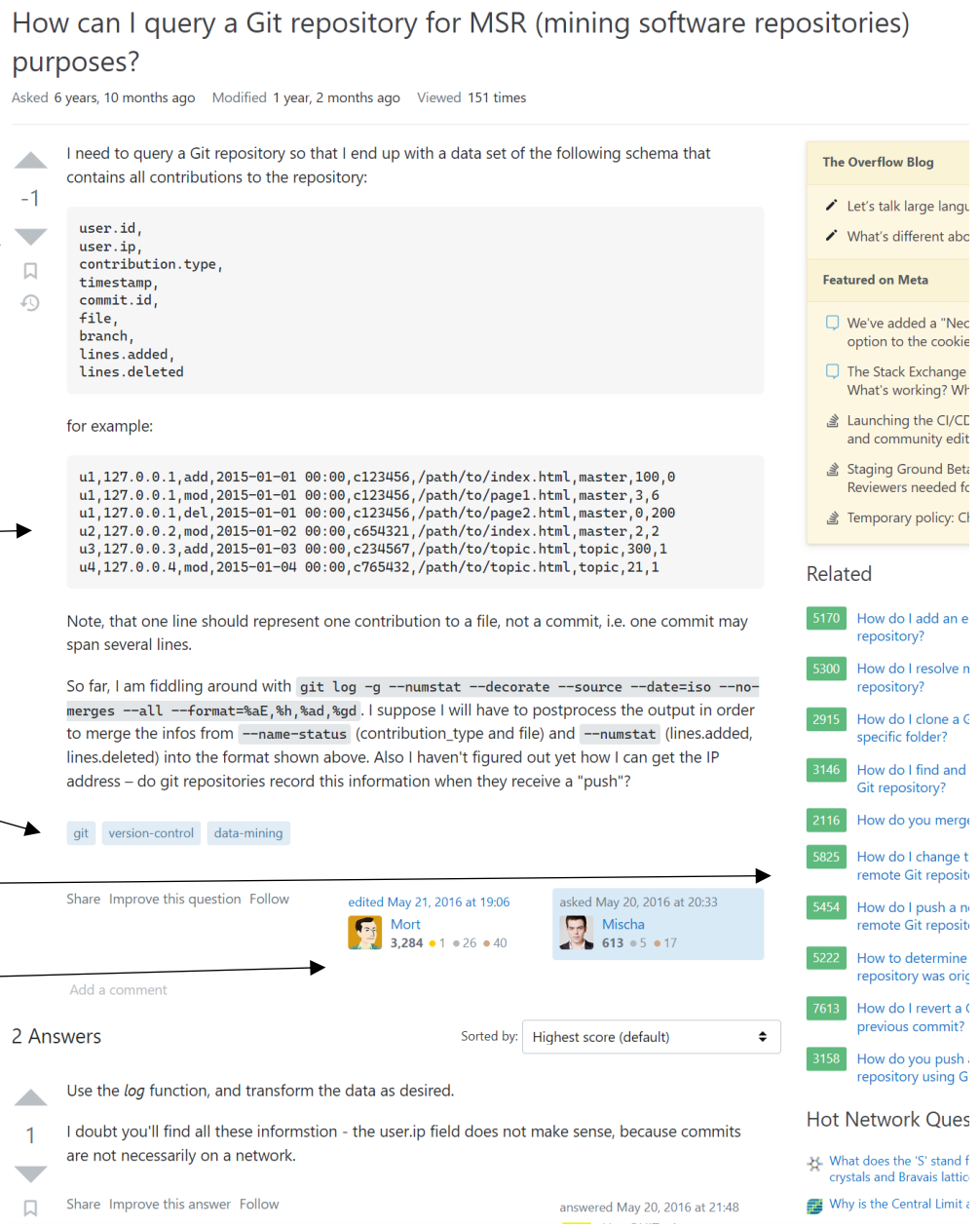
Tags describing the question

Related questions

Users incl. their ratings/achievements

List of answers

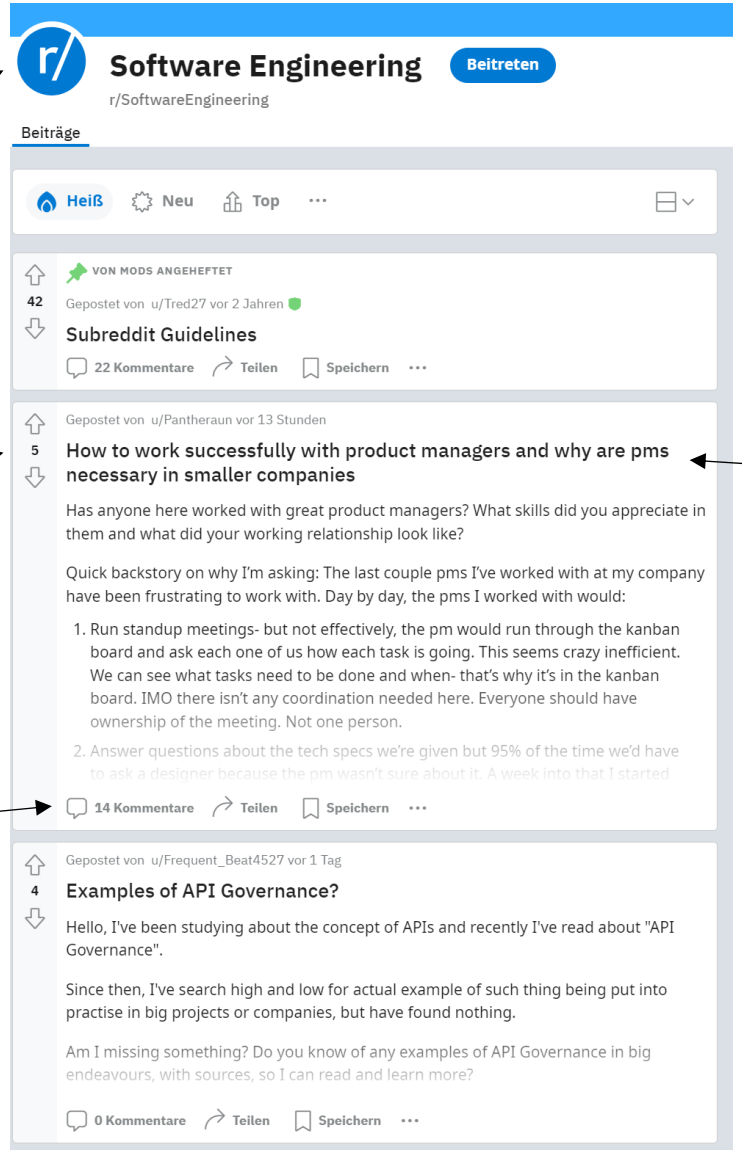
Votes for the answer



https://stackoverflow.com/questions/37355410/how-can-i-query-a-git-repository-for-msr-mining-software-repositories-purposes

Reddit

Different subreddits for different topics



Votes on the topic

Title describing the topic

Description of the topic. Often just links.

Comments organized as a tree. Comments are also voted on.

Also similar portals that are dedicated only to computer science, e.g., HackerNews

https://www.reddit.com/r/SoftwareEngineering/

Community pulse

- Forums and Q&A sites directly show what developers care about
  - What questions are they answering?
  - What topics are they discussing?
  - What are they thoughts and solutions?
- Good source to study trends
  - Which technologies are trending?
  - Which communities interact with each other?
- Good source to study solutions suggested by developers and voted for as good by the community

Tools for data collection

Types of data collection tools

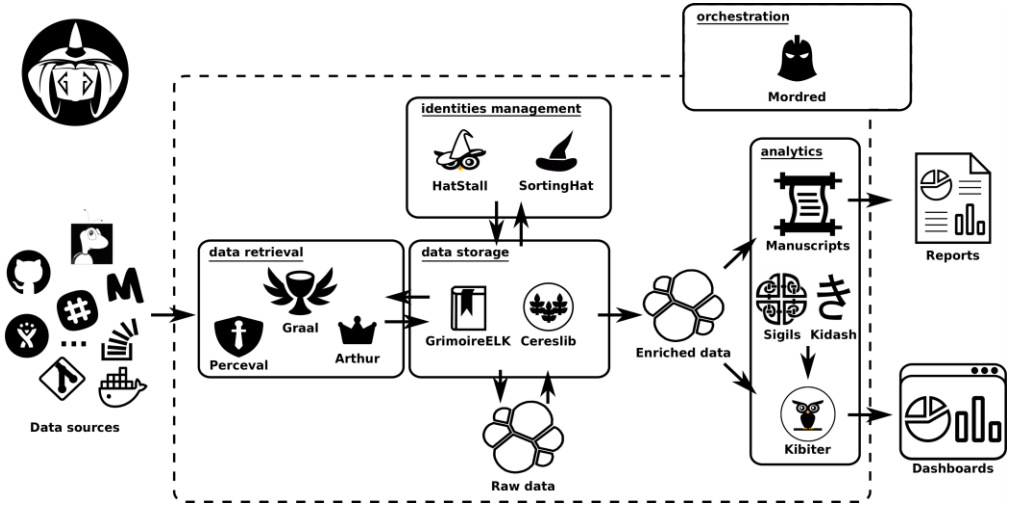
- Off-the-shelf tools
  - MSR tool suites
    - Tools build to collect rich data from different sources
    - Multi purpose tools that enable different kinds of research and applications
  - Single purpose tools
    - Tools build to collect from a single repository often with a specific focus
- Self-made tools
  - Tools created as part of a specific MSR study/application to collect the required data
  - Focus often not on re-use
- Same as for data sources: we now show a non-exhaustive list of examples

Examples for existing tools

Grimoire Labs (former Metrics Grimoire)

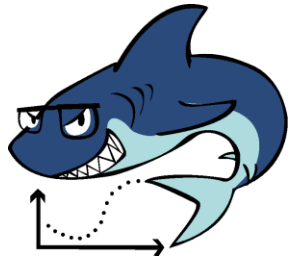


- Tools for the collection of almost all data sources presented
- Data collection fetches data from repositories as JSON and stores this in a large raw data base
  - Dedicated command line tools for each data source
  - Each tool gets its own JSON format with the fields from the tool
  - Facilitates the addition of new data collection tools
- Technologies based on, e.g., ElasticSearch are then used to extract data from this large JSON store
- Also contains components for dashboards, identity disambiguation, and orchestration of the data collection
- Development supported by Researchers, the Linux Foundation, and the company Bitergia

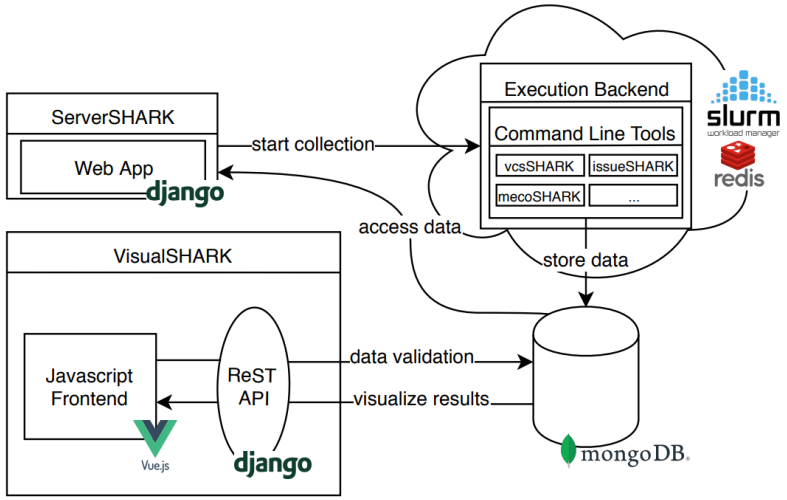


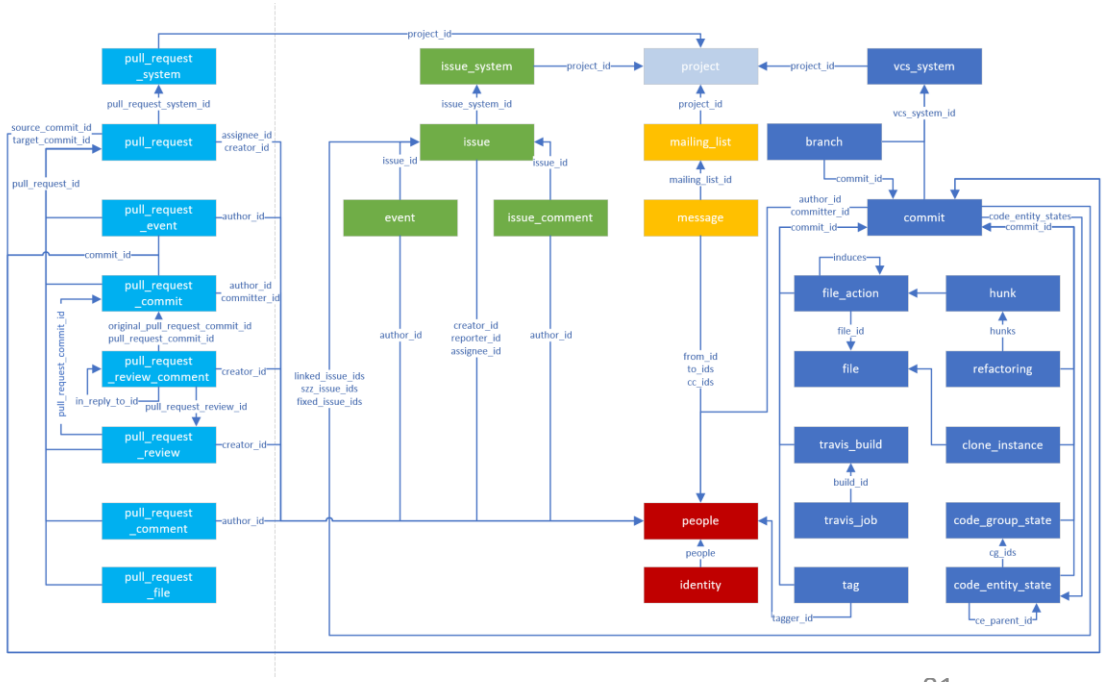
https://chaoss.github.io/grimoirelab/

SmartSHARK



- Similar structure to Grimoire Labs
  - Command line tools for data collection
  - Orchestration tools to handle data collection
  - JSON storage for collected data (MongoDB)
  - Some dashboards available
- Major differences in the details:
  - One common database schema
    - Maps fields with same semantics to the same data base fields
    - Example: Bug ID from Bugzilla and Issue ID from Jira provide the same information
    - Mapped to the same fields
    - Analysis scripts more independent from data source
  - Allows manual validation of data
    - Grimoire Labs allows only collection from repositories
    - SmartSHARK allows research to manually validate data
  - Links between systems
    - Grimoire Labs treats all systems independently
    - SmartSHARK establishes links, e.g., between issues and the commits in which they were addressed





PyDriller

- PyDriller
  - Python tool that fetches information from Git
  - Simple API that enables convenient access to often required data

```
for commit in Repository("/Users/dspadini/myrepo").traverse_commits():
    print(commit.hash)
```

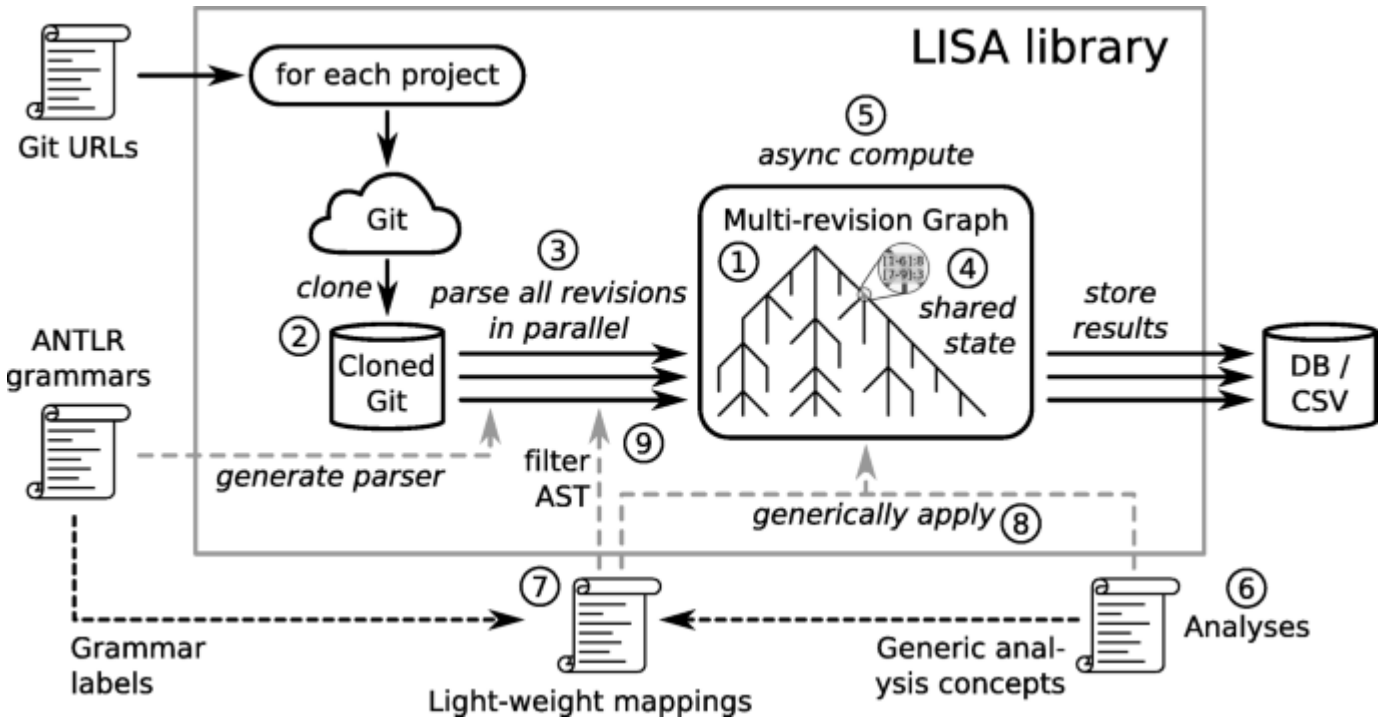
  - Commit graph, messages, history, source code and diffs
  - Can also compute some simple code metrics

- Easy to use Python API makes this a popular choice for anything that only requires metadata from Git

Spadini, Davide, Maurício Aniche, and Alberto Bacchelli. "Pydriller: Python framework for mining software repositories." *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2018.

LISA

- Collecting data from ASTs over thousand of revisions time intensive
  - Building AST is expensive
- LISA can speed this up by building the AST in parallel for many revisions
  - Saves compute time
- Can be used to extract static analysis data
  - However: no dependency information



Alexandru, C.V., Panichella, S., Proksch, S. et al. Redundancy-free analysis of multi-revision software artifacts. Empir Software Eng 24, 332–380 (2019). https://doi.org/10.1007/s10664-018-9630-9

Tools for static source code analysis

- Collect data about code
  - Often need to be integrated into a larger MSR tool chain, e.g., collection for different project or different commits
- Every compiler and parser
  - Sometimes already perform sophisticated static analysis, often resulting in warnings
- Specialized linters, e.g.,
  - pylint to check for coding standards and code smells in python
  - Checkstyle, PMD, or similar aspects for Java
- Powerful suites that consider many aspects, e.g.,
  - SonarCube that supports many languages and types of analysis and warnings
  - LGTM that is integrated in GitHub
- Measurement of software metrics
  - SLOCCount that measures the size
  - CKJM that captures object-oriented metrics

Building your own tools

Using APIs of Web services

- Many tools run as a Web service

- Data can often be retrieved directly via the API, e.g.,
  - GitHub API: https://docs.github.com/en/rest
  - Jira API: https://developer.atlassian.com/server/jira/platform/rest-apis/
  - Bugzilla API: https://wiki.mozilla.org/Bugzilla:REST_API
  - Gerrit API: https://gerrit-review.googlesource.com/Documentation/rest-api.html
  - …

- Requires writing own tools for querying API and processing responses
  - Needs to consider aspects like access rights, pagination, and rate limits

Web scraping

- Tools that automatically traverse Web pages and collect information

- Web scrapers can become the core of a data collection tool
  - E.g., to collect data from a list of the home pages of software projects.
- Method to collect data from Web sites about software

- Typically not necessary and avoided:
  - Most repositories APIs or regular dumps of the database
  - Most information from Websites is also contained elsewhere
  - Scraping usually comes with a lot of noise and requires a huge amount of data cleaning

  → Explore other options before using Web scraping

Custom code analysis tools

- MSR of code often closely related to work on static analysis

- May require building own static analysis tools, e.g.,
  - to scale to hundreds of thousand of version (similar to LISA)
  - because desired property not found in off-the-shelf tool
  - because existing tools only work for current versions, but not a whole repository

- Trade-offs to consider
  - Support for legacy code:
    - Collecting data from old code may require working with old versions of libraries and programming languages → large effort
    - Collection only from the happy path of easy to analyze data may bias results → large risk
  - Scaling to large mounts of data:
    - Optimizing for runtime often time consuming, difficult, and fault-prone → large effort
    - Large amounts of compute resources can often make up for a lack of optimization → requires availability (and configuration) of suitable hardware
  - Accuracy of analysis:
    - The most accurate and best suitable information is often hard to obtain → large effort
    - Similar information and proxy may be less powerful, but might be easier to obtain → some risk (depends)

Reusing data

Data available → No need for collection!

- MSR community usually shares data
  - Supports reproduction and replication of research
- Some companies support this actively
  - E.g., by regularly providing their data for research
- Re-use has benefits
  - Less effort for data preperation means more time for analysis
  - Leads to a better comparability between research results
- Re-use has risks
  - Problems with data may affect everybody that re-used the data
  - Might not be the data that is required, but the data that is convenient

People and companies share data!

- SmartSHARK: https://smartshark.github.io/
  - Code, VCS, Issues, Pull Requests and CI data for over 100 projects
- GitHub Archive: https://www.gharchive.org/
  - Records from GitHub available on Google BigQuery
- Stack Exchange data dump: https://archive.org/details/stackexchange
  - Complete backup of the stack exchange data (incl. Stack Overflow)
- Awesome MSR: https://github.com/dspinellis/awesome-msr
  - Curated collection of data sets (and tools, papers)
- SEACRAFT repository: https://zenodo.org/communities/seacraft
  - Zenodo organization to aid finding archived data sets
- MSR data track
  - Publishes tens of data sets every year – in addition to what is published together with technical papers
  - E.g., for 2023: https://conf.researchr.org/track/msr-2023/msr-2023-data-showcase

Questions?

